

Monitoring of wastewater treatment plants using neural networks

by

Jacob Johannes Mostert

Thesis presented in partial fulfilment of the requirements for the degree of Doctor of Philosophy (Electronic Engineering) in the Faculty of Engineering at Stellenbosch University

Supervisor: Prof. Thomas Niesler

2026

Someone.

Acknowledgements

* Supervisor X.
* Funding Y.

> *A very long quote*
> *from someone very important.*
> – Mr M

Thank you to Leanne for helping to prepare this template.

Declaration

By submitting this thesis electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: 10 May 2024

Copyright © 2026 Stellenbosch University
All rights reserved.

Abstract

Uittreksel

Table of Contents

Abstract iii

Uittreksel iv

Table of contents v

List of figures viii

List of tables ix

Nomenclature xiv

Acronyms and abbreviations xv

1. Introduction 1

1.1. Problem Statement 1

1.2. Objectives 1

1.3. Scope 2

1.4. Report Structure 2

2. Background 3

2.1. Wastewater Treatment Works 3

2.2. Convolutional Neural Networks 3

2.2.1. Building Blocks of a Convolutional Neural Network 3

2.2.2. Residual Networks: ResNet-18 6

2.2.3. Densely Connected Networks: DenseNet-121 7

2.2.4. Efficient Networks: EfficientNet-B0 9

2.2.5. ConvNeXt-Tiny 10

2.2.6. Summary of Architectures 11

2.3. Evaluation Metrics for Classification 11

2.4. Gradient-Weighted Class Activation Mapping 12

2.5. Conclusion 13

3. Literature Review 14

3.1. The Monitoring Problem in South Africa 14

3.2. Reading Process State from Images Inside a Plant 14

3.3. Satellite Remote Sensing of Water Quality 15

3.4. Remote Sensing of Treatment Plants 15

3.5. Pretrained CNNs on a Small, Texture-Defined Problem 16

3.6. Testing on Unseen Plants and Understnading What the Model Sees . . . 16
3.7. Summary and Conclusion . . . 17

4. Dataset **18**
4.1. WWTW Sites . . . 18
4.2. Annotations . . . 19
4.3. Facility Structure . . . 20
4.4. Data Preparation . . . 21
4.5. Excluded from the dataset . . . 22
4.6. Data Splitting . . . 22
4.6.1. Leave-one-out Split . . . 22
4.6.2. Chronological Split . . . 23
4.7. Relabelling . . . 23
4.8. Final Dataset . . . 24
4.9. Conclusion . . . 24

5. Experimental Design **26**
5.1. Overview . . . 26
5.2. Model Architectures and Transfer Learning . . . 27
5.3. Augmentation . . . 28
5.4. Class Imbalance . . . 28
5.5. Hyperparameter Tuning . . . 30
5.5.1. Why Optuna . . . 30
5.5.2. Search Procedure . . . 31
5.5.3. ResNet-18 (Scratch) . . . 31
5.6. Training Procedure . . . 32
5.7. Experiment 1: Mixed-Facility Baseline . . . 33
5.8. Experiment 2: Leave-One-Facility-Out Evaluation . . . 34
5.9. Experiment 3: Binary (OK / Not-OK) Reformulation . . . 36
5.10. Evaluation Metrics . . . 37
5.11. Qualitative Analysis: Grad-CAM . . . 37
5.11.1. Method . . . 37
5.11.2. Models and Example Selection . . . 38

6. Results **39**
6.1. Overview . . . 39
6.2. Multi-Class Classification . . . 39
6.3. Binary (OK / Not-OK) Classification . . . 41
6.4. Qualitative Analysis: Grad-CAM . . . 42
6.4.1. Clarifiers . . . 42

6.4.2. Aerobic Zones 44
6.4.3. Summary of Observations 45
6.5. Summary of Findings 45

7. Summary and conclusions **46**

References **47**

A. Dataset Details **52**
A.1. Annotation Format 52
A.2. Leave-one-out Split 52
A.3. Relabelling Interface 52

B. Hyperparameter Search Trials **54**
B.1. Clarifiers 54
B.1.1. ResNet-18 54
B.1.2. EfficientNet-B0 54
B.1.3. DenseNet-121 55
B.1.4. ConvNeXt-Tiny 56
B.2. Aerobic Zones 56
B.2.1. ResNet-18 56
B.2.2. EfficientNet-B0 57
B.2.3. DenseNet-121 58
B.2.4. ConvNeXt-Tiny 58

C. Additional Results **60**
C.1. Experiment 1: Mixed-Facility Baseline 60
C.2. Experiment 2: Leave-One-Facility-Out Evaluation 62
C.3. Effect of Augmentation 67
C.4. Binary (OK / Not-OK) Results 67

List of Figures

2.1. Examples of functional and dysfunctional wastewater treatment components. 4

4.1. Number of labelled images per month for each of the five facilities. Cape Flats and Waterval have a separate image for each unit on most dates, so their cells can be larger than one. 18
4.2. Location of the five WWTW facilities in the dataset. 19
4.3. Overview of the Cape Flats WWTW facility. 19
4.4. One example of each aerobic zone class, and the facility it comes from. Each example is a test image from the chronological split (Section 5.7) that all four pretrained models classified correctly with the highest mean confidence, so that it is a clear case of its class. 20
4.5. One example of each clarifier class, and the facility it comes from, chosen in the same way as in Figure 4.4. The area outside each tank is masked to black by the preparation script (Section 4.4). 20
4.6. A relabelled clarifier from Cape Flats, shown with some of its surroundings. The original label was “Scum”, but the label was corrected to “Empty”, since the tank appears to be drained, with its floor and the shadow of the bridge visible inside it. 24

5.1. Experimental pipeline, from labelled satellite images to one results file per component, architecture and split. 26
5.2. Augmentations applied to a single **Scum** clarifier from Cape Flats. Top row: the original image, followed by a horizontal flip, a rotation of 15°, and a random colour jitter, each applied on its own. Bottom row: four random outputs of the full augmentation pipeline (flip, rotation and colour jitter combined). 29
5.3. The training loop used for every architecture: an epoch of training and development-set evaluation, a scheduler step, a checkpoint decision based on development loss, and a stopping check repeated every epoch until training ends. 32

6.1. Confusion matrices of the architecture with the highest mean macro F1 score in Experiment 2 for each component, on the held-out facility used for the Grad-CAM analysis (Section 6.4). The confusion matrices of the other folds are given in Appendix C.2. 41

6.2. Grad-CAM heatmaps of the ConvNeXt-Tiny clarifier model, for test images from Fisantekraal, which was held out during training. Top: one correctly classified clarifier per class. Bottom: one clarifier from each of the four most common mistakes. Every heatmap is calculated for the assigned class, and p is the softmax probability of that class. Red regions contributed the most to the classification. 43
6.3. Grad-CAM heatmaps of the DenseNet-121 aerobic zone model, for test images from Waterval, which was held out during training. Top: one correctly classified aerobic zone per class. Bottom: one aerobic zone from each of the three most common mistakes. Every heatmap is calculated for the assigned class, and p is the softmax probability of that class. Red regions contributed the most to the classification. 44

A.1. Number of clarifier images from each facility in the training, development and test sets when Waterval is held out. The test set contains only Waterval, while the remaining four facilities are split into training and development sets. 53
A.2. The relabelling interface, showing the original label and the per-class correction buttons. 53

C.1. Training and development loss and accuracy of the ConvNeXt-Tiny clarifier model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint. 63
C.2. Confusion matrices of the ConvNeXt-Tiny clarifier model on each held-out facility. 64
C.3. Training and development loss and accuracy of the DenseNet-121 aerobic zone model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint. 66
C.4. Confusion matrices of the DenseNet-121 aerobic zone model on each held-out facility. 66

List of Tables

2.1. Structure of ResNet-18 for a 224 × 224 input. Each row is one layer or a group of identical consecutive layers. Kernel gives the dimensions of one kernel, Num the number of kernels, and s and p the stride and padding (Equation 2.2). Shortcut rows are not counted in the 18 layers. In each block, the second ReLU is applied after the shortcut is added. Params includes batch normalisation. The parameter counts in this and the following tables are from torchvision [1], with the 1000-class ImageNet output layer. 7

2.2. Structure of DenseNet-121 for a 224 × 224 input, with columns as in Table 2.1. Each dense layer is a 1 × 1 convolution followed by a 3 × 3 convolution, and a range such as 64–224 means that the number of input channels grows by $k = 32$ with each dense layer. Batch normalisation and ReLU precede each convolution and the global average pool, except for convolution 1, which they follow. 8

2.3. Structure of EfficientNet-B0 for a 224 × 224 input. Each row is one layer, one MBConv block, or a group of identical consecutive blocks. For a block, Kernel and s refer to the depthwise convolution, while Expanded is the number of channels after the expansion. MBConv6 has an expansion ratio of $t = 6$, and MBConv1 has no expansion convolution. For the two ordinary convolutions, the number of kernels equals the number of output channels. 9

2.4. Structure of ConvNeXt-Tiny for a 224×224 input, with columns as in Table 2.3. For a block, Kernel refers to the depthwise convolution and Expanded is the number of channels after the first 1 × 1 convolution. Layer normalisation follows the stem convolution and precedes each downsampling convolution and the FC layer. 10

2.5. Summary of the five models used in this project, ordered by year of publication, with their number of layers and parameter counts in millions. Layers counts the convolutional and fully-connected layers on the main path of each network, so shortcut convolutions and the fully-connected layers inside squeeze-and-excitation blocks are not counted. The clarifier and aerobic zone columns include the tuned classifier head of each final model. 11

4.1. Number of units and labelled images per facility. 21

4.2. Clarifier regions per class, per facility, before (Old) and after (New) relabelling. Relabels is the number of regions whose label changed. 24

4.3. Summary of image counts per class for the Aerobic Zones. 25

4.4. Summary of image counts per class for the Clarifiers. 25

5.1. Parameter count of each of the four architectures evaluated, from smallest to largest, as defined in torchvision with the original 1000-class ImageNet classifier. The new classifier heads (Section 5.5) change these totals. 27
5.2. Number of training images per clarifier class and the resulting class weight w_c, for each held-out test facility and for the chronological split. 30
5.3. Number of training images per aerobic zone class and the resulting class weight w_c, for each held-out test facility and for the chronological split. . . 30
5.4. Search range of each tuned hyperparameter, and the values used for each architecture and component, with the development accuracy of the selected trial. The learning rate is sampled log-uniformly, and the batch size from $\{8, 16, 24, 32, 48, 64, 96, 128\}$. H=0 means the classifier head is a single linear layer, and N is unused. The ResNet-18 (scratch) values were not tuned (Section 5.5.3). 31
5.5. Number of images per class in the training, development and test sets of the chronological split used in Experiment 1. The test set contains every image captured from January 2020 onwards. 33
5.6. Number of training, development and test images in each leave-one-facility-out fold, and the number of test images per class. The test set is the held-out facility, and the training and development sets are drawn from the remaining facilities. 35

6.1. Macro-averaged F1 score and recall of every architecture on the chronological split (Exp. 1) and averaged across held-out facilities (Exp. 2, mean ± standard deviation). Δ is the change in the F1 score from Experiment 1 to Experiment 2, in percentage points. 39
6.2. Recall of every failure class on the chronological split (Exp. 1) and averaged across the held-out facilities at which the class occurs (Exp. 2). 40
6.3. Binary OK / Not-OK results of every architecture on the chronological split (Exp. 1) and averaged across held-out facilities (Exp. 2). The AUC does not depend on a threshold. The recall, specificity and precision of the Not-OK class are given on the test set, at the threshold chosen for a Not-OK recall of 95% on the development set (Section 5.9). The thresholds are listed in Appendix C.4. 41
6.4. Not-OK recall and specificity for each held-out facility at the threshold chosen on the development set of that fold, for the ConvNeXt-Tiny clarifier model and the DenseNet-121 aerobic zone model. The results of every architecture are given in Appendix C.4. 42

B.1. All 20 clarifier Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 54

B.2. All 20 clarifier Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 55
B.3. All 20 clarifier Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 55
B.4. All 20 clarifier Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded. 56
B.5. All 20 aerobic zone Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 57
B.6. All 20 aerobic zone Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 57
B.7. All 20 aerobic zone Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 58
B.8. All 20 aerobic zone Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded. 59

C.1. Clarifier test results of every architecture on the chronological split (Experiment 1). Macro-averaged scores give every class the same weight. The best value in every column is shown in bold. 60
C.2. Per-class precision (P), recall (R) and F1 score of every clarifier architecture on the chronological split (Experiment 1). 60
C.3. Aerobic zone test results of every architecture on the chronological split (Experiment 1). 61
C.4. Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture on the chronological split (Experiment 1). 61
C.5. Clarifier test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the five held-out facilities. 62
C.6. Clarifier test accuracy and mean AUC of every architecture for each held-out facility. 62
C.7. Per-class precision (P), recall (R) and F1 score of every clarifier architecture in Experiment 2, averaged over the held-out facilities at which the class occurs. 63
C.8. Aerobic zone test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the three held-out facilities. 65
C.9. Aerobic zone test accuracy and mean AUC of every architecture for each held-out facility. 65

C.10. Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture in Experiment 2, averaged over the three held-out facilities. 66
C.11. Test results on the chronological split with (Aug.) and without (None) augmentation during training. Δ is the difference in percentage points, where a positive value means that augmentation helped. 67
C.12. Threshold t on the Not-OK score chosen for a 95% recall on the development set, with the test recall of the Not-OK class at that threshold. For the leave-one-facility-out split, every fold has its own threshold, and the threshold and recall are averaged across held-out facilities. The specificity and precision at these thresholds are given in Table 6.3 (Section 6.3). 68
C.13. Recall and specificity of the Not-OK class for each held-out facility at the clarifier threshold chosen on the development set of that fold, with the lowest recall and the standard deviation of the recall across facilities. 68
C.14. Recall and specificity of the Not-OK class for each held-out facility at the aerobic zone threshold chosen on the development set of that fold, with the lowest recall and the standard deviation of the recall across facilities. . . . 69

Nomenclature

Acronyms and Abbreviations

MFT My Funky Thesis

Chapter 1

Introduction

1.1 Problem Statement

Currently, the monitoring of wastewater treatment works (WWTW) is accomplished using on-site sensors, physical inspections, or the manual analysis of aerial photography by experienced personnel. While facilities equipped with sensors connected to automated control systems exist globally, they are currently not widely implemented in South Africa. Consequently, South African facilities rely primarily on on-site personnel to monitor operations. These personnel are required to sample the water at various stages of the treatment process at least once a day in an on-site laboratory, and at least once a month, samples should be sent to an independent laboratory for a complete analysis.

However, one or both of these testing protocols do not always occur. Incorrect measurements may be recorded, or tests may be omitted entirely. To detect such problems, physical site visits by officials are currently necessary. These physical inspections are expensive, time-consuming, and impractical to conduct with high frequency. Furthermore, on-site sensors, where utilised, are prone to failure and expensive to install and maintain.

The question this project aims to answer is whether a computer vision system can accurately identify from aerial or satellite images the same visual indicators utilised by trained human operators (such as the characteristic surface turbulence of an aerator or the effluent clarity of a clarifier) to assess performance. Such a system could be utilised to detect problem cases automatically and quickly. Specifically, a discrepancy between the reported laboratory data and the actual visual state observed in the imagery can be used to trigger corrective action. The envisioned system is a dashboard that classifies new satellite images of each facility and flags components that do not appear to be working. An operator reviews the flagged images and corrects wrong labels, and the corrected images are added to the training data, so that most facilities monitored by the system have labelled images in the training data. Ultimately, this study investigates the feasibility of leveraging historical satellite imagery to automatically monitor the functional state of Fine Bubble Diffused Aeration (FBDA) systems and clarifiers.

1.2 Objectives

To answer the above question, the project addresses the following three objectives:

1. Prepare image datasets for experimentation and extract features by working with a set of labelled Google Earth satellite images taken on different dates to capture the visual states of WWTW components.

2. Train classifiers to automatically determine the status of the aerobic zones (Functional, Suboptimal, Dysfunctional) and the Clarifiers (Functional, Dysfunctional, Scum, Empty), based on the image data.

3. Evaluate the model's performance by testing it on a separate set of images and comparing its classifications to the known labels, using metrics like precision, recall, F1-score, and area under the ROC curve to quantify how well the model performs in classifying the different states of the WWTW components.

1.3 Scope

This project considers the Fine Bubble Diffused Aeration (FBDA) in aerobic zones, and the secondary clarifiers. The project looks at five WWTW facilities: Atlantis, Cape Flats, Waterval, Fisantekraal and Northern Works. Clarifiers are labelled at all five facilities, and aerobic zones only at Atlantis, Cape Flats and Waterval.

Surface-mounted aerators and anaerobic zones are not considered in this project. All images are obtained from Google Earth, and there is no live satellite feed, no physical water sampling, and no hardware devices deployed on site.

1.4 Report Structure

The remainder of the report is structured as follows:

- Chapter 2 (Background): Describes the WWTW components that are classified in this project, and the CNN architectures that are compared.

- Chapter 3 (Literature Review): Describes existing monitoring solutions for wastewater management, remote sensing technology, and the image processing concepts used in this project.

- Chapter 4 (Dataset): Describes the collection and preprocessing of the image datasets.

- Chapter 5 (Experimental Design): Presents a detailed design of the proposed solution, including the convolutional neural network (CNN) architectures compared for classification, the training procedures and the experiments.

- Chapter 6 (Results): Presents the classification performance metrics and discusses the findings, including any limitations or challenges encountered during the experiments.

- Chapter 7 (Conclusion): Draws conclusions from the results, discusses the limitations of the project, and suggests future work that could improve the system.

Background

2.1 Wastewater Treatment Works

Municipal wastewater treatment plants purify our water using a series of steps. Large facilities typically rely on a biological method known as the activated sludge process. This process utilises enormous tanks filled with beneficial microorganisms that naturally break down pollutants. During this process, the microorganisms convert the organic pollutants primarily into carbon dioxide, water, and new cellular biomass. These bacteria require a steady supply of oxygen to survive and work efficiently. To provide this air, facilities often use a technique called Fine Bubble Diffused Aeration (FBDA), which continuously pumps millions of tiny bubbles directly into the water. This happens in a part of the treatment plant known as **aerobic zones**.

Once this active treatment is complete, the mixture flows into large circular settling tanks called **clarifiers**. The dense microbial biomass gradually settles to the bottom of the tank. This settling process leaves clear, treated water at the surface. Rather than discarding this settled biomass, the system continuously circulates it back to the initial tanks. Returning this activated sludge maintains a sufficient concentration of active microorganisms, ensuring the system can efficiently metabolise incoming organic matter.

Interestingly, it is possible to tell a lot about how well these systems are performing just from aerial or satellite photos. A healthy FBDA unit churns the surface into a "leopard print" pattern, while a struggling one looks noticeably smoother. Well functioning clarifiers separate out the solids and leave behind a clear liquid that looks black from above. When they are operating poorly, solids escape into the effluent and the surface turns green, cloudy, or uneven. Shown in Figure 2.1 are examples of functional and dysfunctional aerators and clarifiers, highlighting the visual cues that indicate their performance levels.

2.2 Convolutional Neural Networks

2.2.1 Building Blocks of a Convolutional Neural Network

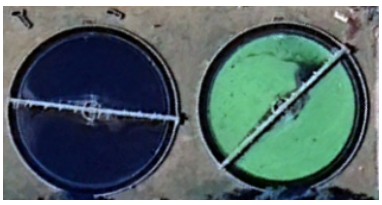
A Convolutional Neural Network (CNN) [2] is a type of artificial neural network designed for data with a grid-like structure, such as the pixel grid of an image. A CNN slides a small filter, or **kernel**, across the image, reusing the same weights at every position. This weight sharing keeps the number of learnable parameters low even for large images, and reflects a reasonable assumption about images of physical objects: a pattern that is useful for identifying, say, the rippled surface of a functioning aerator is just as useful whether it appears in the top-left or bottom-right corner of the image. This makes CNNs particularly



(a) Example of a functional aerator characterised by its "leopard print" pattern.



(b) Example of a dysfunctional aerator with a smooth surface, indicating poor performance.



(c) Functional clarifier and, dysfunctional clarifier. The functional clarifier has a clear surface, while the dysfunctional one has solids floating on top.

Figure 2.1: Examples of functional and dysfunctional wastewater treatment components.

well suited to the classification task in this project, where the relevant visual cues, such as the "leopard print" texture of a functional aerator or the colour and clarity of a clarifier's surface, can appear anywhere within an image and at different scales.

Formally, given an input feature map X with C_{in} channels and a kernel K of size $k \times k$, a convolutional layer computes an output feature map Y where each output value is a weighted sum of a small $k \times k$ neighbourhood of the input, taken across all input channels:

$$Y_{i,j,m} = b_m + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \sum_{c=1}^{C_{in}} K_{u,v,c,m} \, X_{i+u,\, j+v,\, c} \tag{2.1}$$

where:

* i, j index the spatial position (row, column) of the output value being computed;
* m indexes the output channel, i.e. which of the C_{out} kernels produced this value;
* u, v index the position within the $k \times k$ kernel window, ranging over the neighbourhood being summed;
* c indexes the input channel being summed over;
* b_m is a learnable bias term, one per output channel.

In words, $Y_{i,j,m}$ is obtained by placing the kernel so its top-left corner is located at position (i, j) in X, multiplying it element-wise against every input channel underneath it, and summing the result. Each of the C_{out} kernels applied at a layer produces one such output channel, also called a **feature map**, so a convolutional layer with C_{out} kernels turns a C_{in}-channel input into a C_{out}-channel output.

Because the same kernel weights are reused at every position (i, j), a single convolutional layer only ever looks at a $k \times k$ patch of its input. Depth is what lets the network see further: stacking layers causes the **receptive field** (the area of the original image able to influence a given output value) to grow, since a unit in the second layer combines several first-layer outputs, each of which already summarised a $k \times k$ patch of the original image. Two stacked $k \times k$ convolutions therefore give a receptive field of $(2k-1) \times (2k-1)$ pixels, even though neither individual kernel looks beyond its own $k \times k$ window. This is what allows a CNN to build up from simple, local features such as edges and colour gradients in its early layers to increasingly large-scale and abstract patterns, such as the overall texture or shape of a component, in its deeper layers.

Convolutions are usually applied with a **stride** s (the step size the kernel moves between applications) and zero-**padding** p (extra rows and columns of zeros added around the input border, used to maintain the input-output dimensions of the convolution). For a square input of dimensions $W \times W$, the resulting output dimension is

$$O = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1, \tag{2.2}$$

where O is the resulting width and height of the output feature map such that the output feature map has dimensions $O \times O$. A stride greater than one causes the filter to take larger steps accross the input feature map, which shrinks the output feature map and reduces computional work.

After each convolution, an element-wise, non-linear **activation function** is applied to every value of Y. This allows it to learn more complex patterns in the data. All of the architectures used in this project use the Rectified Linear Unit (ReLU), or a smooth variant of it:

$$\mathrm{ReLU}(x) = \max(0, x), \tag{2.3}$$

which simply passes positive input values through unchanged while producing a zero for negative inputs.

Pooling layers are used to progressively reduce the spatial size of the feature maps, most commonly by taking the maximum value within a small patch:

$$Y_{i,j} = \max_{0 \le u,v < k} X_{i \cdot s+u,\ j \cdot s+v}, \tag{2.4}$$

where i, j again index the output position, and u, v index the $k \times k$ pooling window itself, now taken over a single channel at a time rather than summed across channels as in Equation 2.1. Applied with a stride s equal to the window size k, this reduces the resolution of the feature map, reducing the computation required by later layers while making the resulting representation somewhat more robust to small translations of the input. For this project, this means a feature like the “leopard print” texture produces

roughly the same pooled values even when it is located a few pixels off from where it was in another image.

By repeating convolutional, activation, and pooling layers, a CNN gradually trades spatial resolution for channel depth: early feature maps are large and shallow, and later ones are small and deep. Once the feature map is small enough, it is flattened, or globally pooled, into a single feature vector and passed through one or more fully-connected layers, ending in an output layer with one unit per class. A **softmax** function then converts these raw scores, or **logits** z_k, into a probability distribution over the K possible classes:

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad k = 1, \ldots, K, \tag{2.5}$$

where z_k is the logit for class k, p_k is its resulting probability, and the sum in the denominator is over all K classes so that the resulting values of p_k sum to one. The classification decision is the class assigned the highest probability, and the network is trained by adjusting every learnable kernel and weight to make p_k as close as possible to 1 for the correct class, most commonly via the cross-entropy loss.

Training a CNN's convolutional layers entirely from random weight initialisations typically requires very large labelled datasets, since the network has to learn everything from edge detection upward using only the available training images. This project instead relies on **transfer learning**: each architecture is first pretrained on ImageNet [3, 4], a benchmark dataset consisting of over a million labelled natural images spanning a thousand object classes, so that its convolutional layers already encode a broad range of general-purpose visual features before ever seeing a wastewater treatment image. Its final classification layer is then removed and replaced with a small classifier head fitted to the classes used in this project, and the network is fine-tuned on the WWTW image dataset. Four such pretrained architectures, all originally developed for the ImageNet benchmark, were evaluated in this project: **ResNet-18** [5], **DenseNet-121** [6], **EfficientNet-B0** [7], and **ConvNeXt-Tiny** [8]. ResNet-18 is also trained a second time from randomly initialised weights, as a baseline that measures the effect of pretraining (Section 5.2). Each combines the convolutional building blocks above in a different way, as described in the remainder of this section.

2.2.2 Residual Networks: ResNet-18

Residual Networks (ResNet) [5] are built from **residual blocks**. Instead of learning a completely new mapping, the layers of a block learn a residual $F(x)$, and a **shortcut**, or **skip**, connection bypasses the input x around the layers and adds it to their output:

$$y = F(x) + x. \tag{2.6}$$

In traditional networks, the gradient weakens exponentially as it backproagates through each layer, making it difficult to train very deep networks. The skip connection allows the gradient to flow to earlier layers during backpropagation.

ResNet-18 uses a **basic block** of two 3×3 convolutions, each followed by batch normalisation and ReLU, except that the second ReLU is applied after the shortcut is added. Deeper variants such as ResNet-50 use a **bottleneck block**, in which a 1×1 convolution reduces the number of channels, a 3×3 convolution operates on the reduced representation, and a second 1×1 convolution restores the channel count.

Table 2.1 lists the layers of ResNet-18. The 18 counts the layers with weights: 17 convolutional layers and one fully-connected layer. The first convolution of each of the last three stages has a stride of 2, which halves the resolution, while the number of kernels doubles. This means that x and $F(x)$ then no longer have the same shape, so a 1×1 convolution with stride 2, followed by batch normalisation, is applied on the shortcut to make the shapes match for the element-wise addition.

2.2.3 Densely Connected Networks: DenseNet-121

DenseNet [6] extends the shortcut idea of ResNet. Within a **dense block**, each layer receives the feature maps of all preceding layers in the block, concatenated along the channel dimension:

$$x_l = H_l\Big([x_0, x_1, \ldots, x_{l-1}]\Big), \tag{2.7}$$

Table 2.1: Structure of ResNet-18 for a 224×224 input. Each row is one layer or a group of identical consecutive layers. Kernel gives the dimensions of one kernel, Num the number of kernels, and s and p the stride and padding (Equation 2.2). Shortcut rows are not counted in the 18 layers. In each block, the second ReLU is applied after the shortcut is added. Params includes batch normalisation. The parameter counts in this and the following tables are from torchvision [1], with the 1000-class ImageNet output layer.

#	Type	Input	Kernel	Num	s	p	Act	Output	Params
1	Conv	$224 \times 224 \times 3$	$7 \times 7 \times 3$	64	2	3	ReLU	$112 \times 112 \times 64$	9.5K
–	Max pool	$112 \times 112 \times 64$	3×3	–	2	1	–	$56 \times 56 \times 64$	0
2–5	Conv	$56 \times 56 \times 64$	$3 \times 3 \times 64$	64	1	1	ReLU	$56 \times 56 \times 64$	148K
6	Conv	$56 \times 56 \times 64$	$3 \times 3 \times 64$	128	2	1	ReLU	$28 \times 28 \times 128$	74.0K
–	Shortcut	$56 \times 56 \times 64$	$1 \times 1 \times 64$	128	2	0	–	$28 \times 28 \times 128$	8.4K
7–9	Conv	$28 \times 28 \times 128$	$3 \times 3 \times 128$	128	1	1	ReLU	$28 \times 28 \times 128$	443K
10	Conv	$28 \times 28 \times 128$	$3 \times 3 \times 128$	256	2	1	ReLU	$14 \times 14 \times 256$	295K
–	Shortcut	$28 \times 28 \times 128$	$1 \times 1 \times 128$	256	2	0	–	$14 \times 14 \times 256$	33.3K
11–13	Conv	$14 \times 14 \times 256$	$3 \times 3 \times 256$	256	1	1	ReLU	$14 \times 14 \times 256$	1.77M
14	Conv	$14 \times 14 \times 256$	$3 \times 3 \times 256$	512	2	1	ReLU	$7 \times 7 \times 512$	1.18M
–	Shortcut	$14 \times 14 \times 256$	$1 \times 1 \times 256$	512	2	0	–	$7 \times 7 \times 512$	132K
15–17	Conv	$7 \times 7 \times 512$	$3 \times 3 \times 512$	512	1	1	ReLU	$7 \times 7 \times 512$	7.08M
–	Global avg pool	$7 \times 7 \times 512$	7×7	–	–	–	–	512	0
18	FC	512	–	1000	–	–	Softmax	1000	513K
Total									11.7M

where x_0 is the input to the block and x_l the output of layer l. Because the inputs are concatenated rather than added as in Equation 2.6, each layer contributes k new channels to the block, where k is called the **growth rate**. If the input x_0 to a block has k_0 channels, the input to layer l therefore has $k_0 + k(l-1)$ channels. We call each such layer a **dense layer**. In DenseNet-121, $k = 32$ and H_l consists of two convolutions, each preceded by batch normalisation and a ReLU: a 1×1 convolution that produces $4k$ channels, followed by a 3×3 convolution that produces the k new channels.

Between dense blocks, a **transition layer** halves the number of channels with a 1×1 convolution and halves the resolution with a 2×2 average pool. DenseNet-121 has four dense blocks with 6, 12, 24 and 16 dense layers respectively (Table 2.2). Each dense layer contains two convolutions, so the blocks contribute $2(6 + 12 + 24 + 16) = 116$ convolutions. The initial convolution, the three transition convolutions and the fully-connected layer bring the total to the 121 layers. Because each layer can reuse the features of all earlier layers in its block, every layer can be narrow. The network has 8.0M parameters, fewer than the 11.7M of ResNet-18.

Table 2.2: Structure of DenseNet-121 for a 224×224 input, with columns as in Table 2.1. Each dense layer is a 1×1 convolution followed by a 3×3 convolution, and a range such as 64–224 means that the number of input channels grows by $k = 32$ with each dense layer. Batch normalisation and ReLU precede each convolution and the global average pool, except for convolution 1, which they follow.

#	Type	Input	Kernel	Num	s	p	Output	Params
1	Conv	$224 \times 224 \times 3$	$7 \times 7 \times 3$	64	2	3	$112 \times 112 \times 64$	9.5K
–	Max pool	$112 \times 112 \times 64$	3×3	–	2	1	$56 \times 56 \times 64$	0
2–13	Dense	$56 \times 56 \times$ (64–224)	$1 \times 1 \times$ (64–224)	128	1	0	$56 \times 56 \times 128$	112K
	Dense	$56 \times 56 \times 128$	$3 \times 3 \times 128$	32	1	1	$56 \times 56 \times 32$	223K
14	Transition	$56 \times 56 \times 256$	$1 \times 1 \times 256$	128	1	0	$56 \times 56 \times 128$	33.3K
–	Avg pool	$56 \times 56 \times 128$	2×2	–	2	0	$28 \times 28 \times 128$	0
15–38	Dense	$28 \times 28 \times$ (128–480)	$1 \times 1 \times$ (128–480)	128	1	0	$28 \times 28 \times 128$	474K
	Dense	$28 \times 28 \times 128$	$3 \times 3 \times 128$	32	1	1	$28 \times 28 \times 32$	445K
39	Transition	$28 \times 28 \times 512$	$1 \times 1 \times 512$	256	1	0	$28 \times 28 \times 256$	132K
–	Avg pool	$28 \times 28 \times 256$	2×2	–	2	0	$14 \times 14 \times 256$	0
40–87	Dense	$14 \times 14 \times$ (256–992)	$1 \times 1 \times$ (256–992)	128	1	0	$14 \times 14 \times 128$	1.95M
	Dense	$14 \times 14 \times 128$	$3 \times 3 \times 128$	32	1	1	$14 \times 14 \times 32$	891K
88	Transition	$14 \times 14 \times 1024$	$1 \times 1 \times 1024$	512	1	0	$14 \times 14 \times 512$	526K
–	Avg pool	$14 \times 14 \times 512$	2×2	–	2	0	$7 \times 7 \times 512$	0
89–120	Dense	$7 \times 7 \times$ (512–992)	$1 \times 1 \times$ (512–992)	128	1	0	$7 \times 7 \times 128$	1.56M
	Dense	$7 \times 7 \times 128$	$3 \times 3 \times 128$	32	1	1	$7 \times 7 \times 32$	594K
–	Global avg pool	$7 \times 7 \times 1024$	7×7	–	–	–	1024	2.0K
121	FC	1024	–	1000	–	–	1000	1.03M
Total								8.0M

2.2.4 Efficient Networks: EfficientNet-B0

EfficientNet [7] is built using **mobile inverted bottleneck convolution** (MBConv) blocks, which split a full convolution into cheaper steps. First, a 1×1 convolution increases the number of channels by a factor t. Second, a $k \times k$ **depthwise** convolution looks for spatial patterns, such as edges and texture, in each channel separately. Because it does not combine channels, it needs C times fewer multiply-adds than a full convolution with C input and C output channels (Equation 2.1). Third, a **squeeze-and-excitation** block [9] weights each channel by its importance for the current image. To compute this importance, the average of the feature map in each channel is computed and passed through two small fully-connected layers, the second of which ends in a sigmoid. The resulting outputs, which lie between 0 and 1, are viewed as importance values and are used to multiplicatively scale the feature map of each channel. Finally, a 1×1 convolution combines the channels and reduces their number again. The depthwise convolution therefore finds spatial patterns, while the 1×1 convolutions mix the channels. When the input and output of a block have the same shape, the input is added to the output through a residual connection, as in ResNet (Equation 2.6). The activation is SiLU, $\mathrm{SiLU}(x) = x\sigma(x)$, where σ is the sigmoid function, which is a smooth variant of ReLU.

Table 2.3: Structure of EfficientNet-B0 for a 224×224 input. Each row is one layer, one MBConv block, or a group of identical consecutive blocks. For a block, Kernel and s refer to the depthwise convolution, while Expanded is the number of channels after the expansion. MBConv6 has an expansion ratio of $t = 6$, and MBConv1 has no expansion convolution. For the two ordinary convolutions, the number of kernels equals the number of output channels.

Block	Type	Input	Kernel	Expanded	s	Output	Params
–	Conv	$224 \times 224 \times 3$	$3 \times 3 \times 3$	–	2	$112 \times 112 \times 32$	0.9K
1	MBConv1	$112 \times 112 \times 32$	3×3	32	1	$112 \times 112 \times 16$	1.4K
2	MBConv6	$112 \times 112 \times 16$	3×3	96	2	$56 \times 56 \times 24$	6.0K
3	MBConv6	$56 \times 56 \times 24$	3×3	144	1	$56 \times 56 \times 24$	10.7K
4	MBConv6	$56 \times 56 \times 24$	5×5	144	2	$28 \times 28 \times 40$	15.4K
5	MBConv6	$28 \times 28 \times 40$	5×5	240	1	$28 \times 28 \times 40$	31.3K
6	MBConv6	$28 \times 28 \times 40$	3×3	240	2	$14 \times 14 \times 80$	37.1K
7–8	MBConv6	$14 \times 14 \times 80$	3×3	480	1	$14 \times 14 \times 80$	206K
9	MBConv6	$14 \times 14 \times 80$	5×5	480	1	$14 \times 14 \times 112$	126K
10–11	MBConv6	$14 \times 14 \times 112$	5×5	672	1	$14 \times 14 \times 112$	417K
12	MBConv6	$14 \times 14 \times 112$	5×5	672	2	$7 \times 7 \times 192$	262K
13–15	MBConv6	$7 \times 7 \times 192$	5×5	1152	1	$7 \times 7 \times 192$	1.76M
16	MBConv6	$7 \times 7 \times 192$	3×3	1152	1	$7 \times 7 \times 320$	717K
–	Conv	$7 \times 7 \times 320$	$1 \times 1 \times 320$	–	1	$7 \times 7 \times 1280$	412K
–	Global avg pool	$7 \times 7 \times 1280$	7×7	–	–	1280	0
–	FC	1280	–	–	–	1000	1.28M
Total							5.3M

2.2.5 ConvNeXt-Tiny

ConvNeXt [8] starts from a standard ResNet-50 and changes it step by step to resemble a Vision Transformer, a newer type of network that splits the image into small patches, while still using only convolutions. The ConvNeXt block has three steps. First, a 7×7 depthwise convolution looks for spatial patterns in each channel separately. The large kernel lets the block see a wider area of the image, and because the convolution is depthwise, it remains cheap. Second, a 1×1 convolution increases the number of channels by a factor of 4, and is followed by the GELU activation. Third, a second 1×1 convolution reduces the number of channels back to the original number. The input of the block is then added to its output through a residual connection, as in ResNet (Equation 2.6). The block is therefore narrow at its ends and wide in the middle, which is the opposite of the ResNet bottleneck block (Section 2.2.2) but similar to MBConv (Section 2.2.4). The block uses **layer normalisation**, which normalises over the channels of each image instead of over a batch of images, and has only one normalisation and one activation, where the ResNet bottleneck block has three of each. GELU (Gaussian Error Linear Unit) is a smooth variant of ReLU [10]:

$$\mathrm{GELU}(x) = x \cdot \Phi(x), \tag{2.8}$$

where Φ is the cumulative distribution function of the standard normal distribution. At the input, the 7×7 convolution and max pool of ResNet are replaced by a 4×4 convolution with stride 4, called a **patchify** layer, which divides the image into non-overlapping 4×4 patches. Between stages, the resolution is halved by a separate 2×2 convolution with stride 2. ConvNeXt-Tiny has four stages of 3, 3, 9 and 3 blocks with 96, 192, 384 and 768 channels (Table 2.4).

Table 2.4: Structure of ConvNeXt-Tiny for a 224×224 input, with columns as in Table 2.3. For a block, Kernel refers to the depthwise convolution and Expanded is the number of channels after the first 1×1 convolution. Layer normalisation follows the stem convolution and precedes each downsampling convolution and the FC layer.

Block	Type	Input	Kernel	Expanded	s	Output	Params
–	Stem conv	$224 \times 224 \times 3$	$4 \times 4 \times 3$	–	4	$56 \times 56 \times 96$	4.9K
1–3	ConvNeXt block	$56 \times 56 \times 96$	7×7	384	1	$56 \times 56 \times 96$	238K
–	Downsample conv	$56 \times 56 \times 96$	$2 \times 2 \times 96$	–	2	$28 \times 28 \times 192$	74.1K
4–6	ConvNeXt block	$28 \times 28 \times 192$	7×7	768	1	$28 \times 28 \times 192$	918K
–	Downsample conv	$28 \times 28 \times 192$	$2 \times 2 \times 192$	–	2	$14 \times 14 \times 384$	296K
7–15	ConvNeXt block	$14 \times 14 \times 384$	7×7	1536	1	$14 \times 14 \times 384$	10.8M
–	Downsample conv	$14 \times 14 \times 384$	$2 \times 2 \times 384$	–	2	$7 \times 7 \times 768$	1.18M
16–18	ConvNeXt block	$7 \times 7 \times 768$	7×7	3072	1	$7 \times 7 \times 768$	14.3M
–	Global avg pool	$7 \times 7 \times 768$	7×7	–	–	768	0
–	FC	768	–	–	–	1000	771K
Total							28.6M

2.2.6 Summary of Architectures

Table 2.5 summarises the architectures used in this project. Besides the four pretrained architectures described above, it includes ResNet-18 (scratch), the ResNet-18 that is trained from randomly initialised weights as a baseline (Section 5.2). The ImageNet column gives each architecture's size as defined in torchvision, when including its original 1000-class output layer. The last two columns give the size of the final models trained in this project, after that output layer has been replaced with the tuned classifier head of each component (Section 5.5).

Table 2.5: Summary of the five models used in this project, ordered by year of publication, with their number of layers and parameter counts in millions. Layers counts the convolutional and fully-connected layers on the main path of each network, so shortcut convolutions and the fully-connected layers inside squeeze-and-excitation blocks are not counted. The clarifier and aerobic zone columns include the tuned classifier head of each final model.

				Parameters		
Architecture	**Building block**	**Year**	**Layers**	**ImageNet**	**Clarifier**	**Aerobic**
ResNet-18	Basic residual block	2016	18	11.7M	11.4M	11.2M
ResNet-18 (scratch)	Basic residual block	2016	18	11.7M	13.8M	13.8M
DenseNet-121	Dense block	2017	121	8.0M	13.3M	7.0M
EfficientNet-B0	MBConv	2019	50	5.3M	4.0M	10.8M
ConvNeXt-Tiny	ConvNeXt block	2022	59	28.6M	28.6M	33.6M

Table 2.5 shows that a network's parameter count says little about its depth: DenseNet-121, with 121 layers, is one of the smallest. The classifier head can also change a model's size considerably. The head of the EfficientNet-B0 aerobic zone model adds about 6.8M parameters to a backbone of about 4.0M, more than doubling its size.

2.3 Evaluation Metrics for Classification

The performance of a classifier can be measured in several ways.

Accuracy is the fraction of test images that are assigned to their true class. When the classes are imbalanced, it is dominated by the largest classes.

Precision, recall and F1 score are calculated for each class c from the number of true positives TP_c, false positives FP_c and false negatives FN_c:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \qquad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \qquad F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \tag{2.9}$$

The macro-averaged values are the unweighted means over the classes, so that every class has the same weight. The **confusion matrix** counts, for every true class, how many images were assigned to each class. It therefore reflects classes that are often confused.

The **receiver operating characteristic (ROC) curve** plots the true positive rate (recall) against the false positive rate as the decision threshold is varied. The **area under the curve (AUC)** is 1 when every positive image receives a higher score than every negative image, and 0.5 for random guessing. For more than two classes, the one-vs-rest AUC of each class can be averaged.

Loss and accuracy curves give the training and development loss and accuracy after every epoch.

2.4 Gradient-Weighted Class Activation Mapping

Gradient-weighted class activation mapping (Grad-CAM) [11] produces a heatmap that shows which regions of an input image contributed the most to the output of a CNN for a given class.

Grad-CAM uses the output of the last convolutional layer, which is the layer before the global pooling and the fully connected classification layers. This output consists of one feature map per channel.Let A^k denote the feature map of channel k, A^k_{ij} the element in row i and column j. A weight α^c_k is calculated for each channel by taking the gradient of the score y^c of class c, which is the logit of that class before the softmax (Equation 2.5), with respect to every element of A^k, and averaging these gradients over the feature map:

$$\alpha^c_k = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A^k_{ij}}, \tag{2.10}$$

where the sums run over all rows i and columns j of the feature map, and Z is the number of elements in the feature map (its height multiplied by its width).

The feature maps of all channels are then multiplied by their weights and summed.Finally only the positive elements of this matrix are kept:

$$L^c = \mathrm{ReLU}\left(\sum_k \alpha^c_k A^k \right). \tag{2.11}$$

The map L^c has the same height and width as the feature maps of the last convolutional layer, and is much smaller than the input image. It is therefore resized to the dimensions of the input image, normalised to the range $[0, 1]$, and superimposed on the image as a heatmap.

Because the weights α^c_k depend on c, a separate heatmap is obtained for each class $c = 1, \ldots, K$ from the same image. The heatmap of class c highlights the regions whose features increase the score y^c. Regions that lower this score are set to zero by the ReLU. Comparing the heatmaps of different classes therefore shows which parts of the image the network associates with each class, and whether a prediction is based on the relevant object or on unrelated parts of the image.

2.5 Conclusion

In this chapter, we described the activated sludge process and how the aerobic zones and clarifiers appear from above, the building blocks of CNNs and the four architectures used in this project, the metrics used to evaluate them, and Grad-CAM. In the next chapter, we review previous work on judging the state of treatment plants and water bodies from images.

Chapter 3

Literature Review

This chapter reviews the published work that concerns the question: can the state of an aerobic zone or clarifier be judged reliably from historical satellite images, without a site visit? No single field answers it. The relevant studies are spread over four areas, namely image analysis inside treatment plants, satellite remote sensing of water quality, remote sensing of treatment plants, and the behaviour of convolutional neural networks (CNNs) on small and uneven datasets.

3.1 The Monitoring Problem in South Africa

The 2022 Green Drop report for the Department of Public Works shows a clear gap between official records and actual plant conditions [12]. Out of 115 systems, 102 were in a critical state. The report notes a disconnect between administrative data and physical fieldwork. Physical inspections found differences of up to 59% between paper-based scores and the actual state of the facilities. This project proposes using imagery to spot these differences, as the report indicates that scores based on submitted paperwork can hide true performance.

3.2 Reading Process State from Images Inside a Plant

Researchers have long tried to turn an operator's visual judgement of activated sludge into measurement. Quantitative image analysis of microscope images of flocs and filaments has grown into a field of its own [13]. However, each system needs hand-tuned steps such as thresholding, and a practitioner must choose which algorithms and threshold values to use [14]. CNNs remove much of this by learning features from labelled examples. Satoh et al. [15] trained binary CNN classifiers that recognised changes in floc morphology and the abundance of filamentous bacteria. Borzooei et al. [14] predicted the sludge volume index from microscope images taken weekly at a Belgian industrial plant over almost two years. Transfer learning from ImageNet generally offers better performance than training from scratch. Large networks can overfit, and the small ConvNeXt-nano was seen to generalise best. The model of Borzooei et al. [14] also underestimated the highest index values, which the authors attribute to a very small number of such samples in the training set.

In contrast to microscopy, a camera above the tank can also be used. Carballo Mato et al. [16] used images from a pan-tilt-zoom camera at a working plant to segment surface foam with texture segmentation models, reaching a Dice score of 86%, and reported the percentage of the tank surface covered by foam to the operators in real time.

These studies agree that the appearance of sludge and tank surfaces carries real information about process state, and that CNNs can extract it from modest datasets. This supports the premise of this project. The studies by Borzooei et al. [14] and Carballo Mato et al. [16], however, both need sampling or equipment at the plant, which is the cost the problem statement wants to avoid. Borzooei et al. [14] also trained and tested their models on samples from a single plant, so their results do not show whether a model can judge a plant it has not seen.

3.3 Satellite Remote Sensing of Water Quality

Satellite studies estimate suspended solids and turbidity from the light reflected off the water surface, but typically use medium-resolution satellites such as Landsat, whose pixels are tens of metres wide [17]. A clarifier would be resolved as only a few pixels, and the surface pattern of an aeration zone would not be discernible. Google Earth imagery provides far more detail, and is increasingly used to build land cover reference data by visual interpretation [18]. Hu et al. [19] found it almost as accurate as sub-metre multispectral imagery for land cover mapping, but weak on classes that can only be separated by colour, because it carries little spectral information. The classes in this project are of that kind, and no physical quantity such as suspended solids need to be computed from the imagery. The project must therefore learn the visual categories that operators already use.

3.4 Remote Sensing of Treatment Plants

In the remote sensing literature, treatment plants feature mainly as objects to be found. Wastewater treatment plant is one of 38 scene classes in the PatternNet benchmark, with one label per plant whatever its condition [20]. Li et al. [21] combined OpenStreetMap labels with multimodal satellite data to detect plants and tested the method around Stuttgart. Shuai et al. [22] built an attention-based detector for Gaofen-2 imagery that located plants across Beijing with 80.1% accuracy at 90.4% recall, and noted that the mix of rectangular and circular tanks makes plants harder to detect with conventional methods. Randhawa et al. [23] worked towards worldwide detection. In all of this work the tanks help to find a plant, and none of these studies asks whether the tanks are working. The closest regulatory precedent, from agriculture, has the same limit. Handan-Nader and Ho [24] applied a CNN to high-resolution imagery of North Carolina, found 589 more poultry feeding operations than a manual count had (an increase of 15%), and argued that the method can inform compliance priorities. Their target was the location of a facility, not its behaviour.

3.5 Pretrained CNNs on a Small, Texture-Defined Problem

With little labelled data, training a CNN from a random initialisation is usually not practical. Nogueira et al. [25] compared full training, fine-tuning and fixed feature extraction for six CNNs on three remote sensing datasets and found that fine-tuning tended to work best, a result that Borzooei et al. [14] confirm in a very different domain. This project will therefore focus on fine-tuning pretrained networks, and train only one model (ResNet-18) from scratch as a baseline (Section 5.2). The earlier study by Penatti et al. [26] adds a note of caution. ImageNet features gave the best results on aerial scenes but were beaten by a simple colour descriptor on multispectral scenes of coffee crops. Features learnt from everyday objects may therefore miss a cue when classes differ mainly in colour, as the clarifier classes do.

Texture is the other cue. Geirhos et al. [27] showed that ImageNet-trained CNNs rely on texture far more than people do, which they framed as a weakness for recognising objects. In this project, on the other hand, the bias may help, since a working aeration zone is identified by its surface texture, and Carballo Mato et al. [16] likewise treated tank foam as a texture problem. However, concrete walls, walkways and the rotating arms of clarifiers also have strong textures that a model may learn in place of the water. The four pretrained models compared in this project come from four families published between 2016 and 2022 [5–8], and no comparison of these families on treatment plant or similar imagery was found in the literature. Since Borzooei et al. [14] found that larger models did worse on limited data, the smaller variants were prioritised.

Uneven class distributions add a further problem, because the rare states may be the ones that are most important to find. Buda et al. [28] showed that class imbalance harms CNN performance and that oversampling was the best remedy in almost all their tests. Lin et al. [29] proposed focal loss, which down-weights easy examples so that training focuses on hard ones. Both methods were developed on data with distinct classes, such as handwritten digits and everyday objects. The classes here are graded judgements, defined by thresholds such as a third or two thirds of the surface, so a hard example may be borderline or mislabelled rather than informative. Neither study examined that case. This project uses neither method, and instead weights the cross-entropy loss of each class inversely to its frequency in the training set (Section 5.4).

3.6 Testing on Unseen Plants and Understnading What the Model Sees

Kattenborn et al. [30] demonstrated that how you select test data is critical in remote sensing. When analysing drone imagery, choosing random test images made the model appear to be 28% more accurate than it actually was. This happens because images taken from the same area and at the same time are too similar, giving the model an unfair

advantage. To get a realistic performance score, the model must be tested on entirely separate geographic locations, a rule that becomes even more important when you only have a small amount of data. The data in this project require this to be considered. Each tank appears many times at different dates, so a random split would put images of the same tank, taken months apart, into both the training and test sets.

Inflated scores are not the only risk. Geirhos et al. [31] describe shortcut learning, in which a network finds a rule that works on the test set but fails under new conditions, and Lapuschkin et al. [32] give the well-known case of a classifier that recognised horses by a copyright tag common in horse photographs. With a handful of plants imaged over many years, the equivalent shortcut would be to recognise which plant is in view and how often that plant was labelled as failing. Holding out whole facilities blocks this route. Checking where the model looks is a second safeguard. For example gradient-weighted class activation mapping (Grad-CAM) highlights the image regions that are important for the output of a CNN [11], so it can test whether the model uses the same cues a human operator would use, and not merely whether it reaches the same answer.

Overall accuracy hides poor results on rare classes, which is why Buda et al. [28] used a multi-class version of the area under the receiver operating characteristic (ROC) curve. Saito and Rehmsmeier [33] argue that precision-recall curves are more informative than ROC curves when classes are imbalanced, so the ROC-based AUC used in this project is reported alongside the per-class precision and recall rather than on its own. Cost matters as well. A tool that triggers inspections can accept some false alarms but should rarely miss a failing unit, so recall on the failing classes, at a threshold chosen for that purpose, is the more useful measure here (Section 5.9).

3.7 Summary and Conclusion

This chapter has reviewed the literature relevant to judging the state of aerobic zones and clarifiers from historical satellite images. Image analysis inside treatment plants shows that the appearance of sludge and tank surfaces carries information about process state, but these methods require sampling or equipment at the plant. Satellite studies of water quality use medium-resolution spectral data, which is too coarse for individual tanks, while Google Earth imagery has sufficient resolution but little spectral information. Remote sensing studies of treatment plants address the detection of plants and not their operating state, and no study was found that classifies the state of treatment units from satellite imagery. The literature further supports fine-tuning pretrained CNNs on small datasets, testing on facilities withheld from training, and reporting per-class precision and recall. In the next chapter we will describe the dataset of satellite images used in this project, its annotation and how it was split for training and testing.

Dataset

The dataset used in this project consists of satellite imagery sourced from Google Earth and covers five waste water treatment works (WWTW) facilities: Atlantis, Cape Flats, Waterval, Fisantekraal and Northern Works. Each image is a snapshot of a facility's biological reactors at a particular point in time, and its filename encodes the month and year in which it was captured, although the exact format differs between facilities (for example `Atlantis 01_2019`, `capeflats1_201408` and `Northern Works 2020 05`).

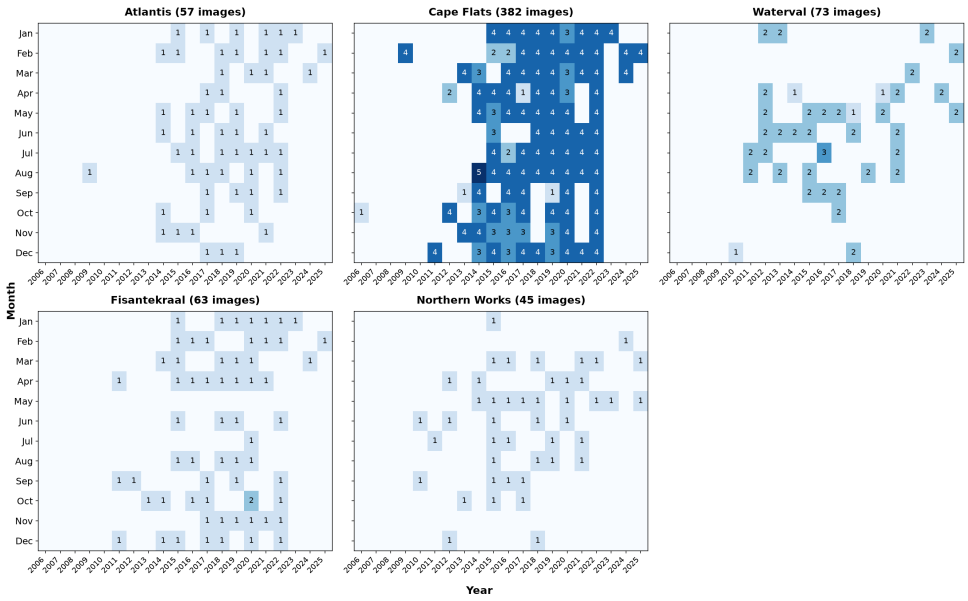


Figure 4.1: Number of labelled images per month for each of the five facilities. Cape Flats and Waterval have a separate image for each unit on most dates, so their cells can be larger than one.

The dataset contains 620 labelled images, captured between October 2006 and May 2025. As seen in Figure 4.1, most of them (423 of the 620) were captured between 2016 and 2022, and they are spread fairly evenly over the months of the year, with between 38 (November) and 62 (August) images per month.

4.1 WWTW Sites

Atlantis, Cape Flats and Fisantekraal WWTW are located in the Western Cape, in and around Cape Town. Waterval WWTW, operated by ERWAT (Ekurhuleni Water Care Company), and Northern Works (Noordelike Werke) in Johannesburg are located in Gauteng. Northern Works is the largest plant in Johannesburg and among the largest in South Africa, treating approximately 400 million litres of wastewater per day. The

locations of the sites are shown in Figure 4.2, and an overview of the Cape Flats facility is shown in Figure 4.3.



Figure 4.2: Location of the five WWTW facilities in the dataset.

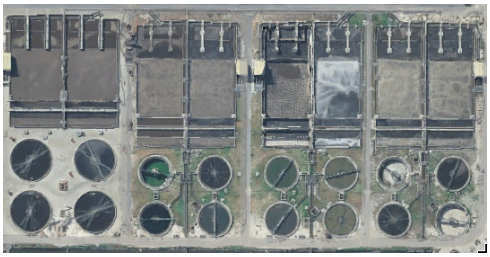


Figure 4.3: Overview of the Cape Flats WWTW facility.

4.2 Annotations

Images were manually annotated using the VGG Image Annotator (VIA2) tool [34]. Aerobic zones were labelled using rectangular bounding boxes, while clarifiers were labelled using circular regions, reflecting the physical shape of each structure. Each annotation carries a region attribute indicating the visible state of that component at the time the image was captured.

Aerobic zones were labelled into one of three categories, based on how much of the surface exhibits the characteristic "leopard print" aeration pattern. Examples are shown in Figure 4.4: **Functional**: the surface shows the leopard print pattern. **Suboptimal**: a third or more of the surface shows no leopard print, or the state is unclear. **Dysfunctional**: two thirds or more of the surface shows no leopard print, or is clearly dysfunctional.

Clarifiers were labelled into one of four categories, based on surface colour and visibility. Examples are shown in Figure 4.5: **Functional**: surface appears uniformly black. **Dysfunctional**: surface appears green. **Scum**: surface is not uniformly black. **Empty**: shadows are visible inside the tank.

Annotations are stored as VIA2 project files in JSON format. Each entry records the image filename, file size, and a list of regions, where each region specifies its shape

(rectangle or circle), pixel coordinates, and the associated class label. An example entry for an aerobic zone annotation is given in Appendix A.

Examples of the classes for aerobic zones and clarifiers are shown in Figures 4.4 and 4.5 respectively.

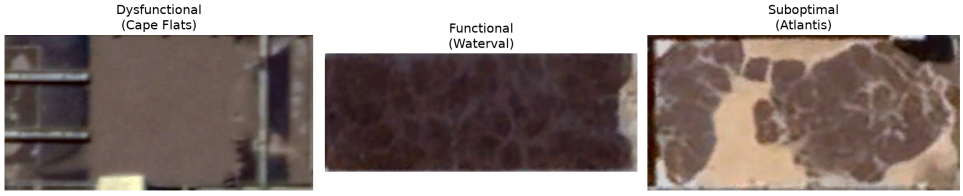


Figure 4.4: One example of each aerobic zone class, and the facility it comes from. Each example is a test image from the chronological split (Section 5.7) that all four pretrained models classified correctly with the highest mean confidence, so that it is a clear case of its class.

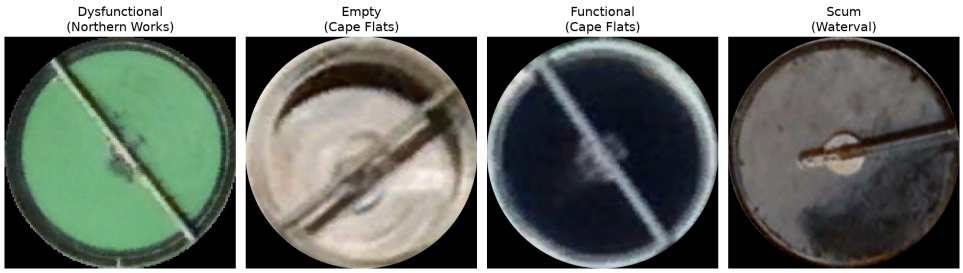


Figure 4.5: One example of each clarifier class, and the facility it comes from, chosen in the same way as in Figure 4.4. The area outside each tank is masked to black by the preparation script (Section 4.4).

4.3 Facility Structure

The facilities differ slightly in how their data is organised, reflecting differences in plant layout. Atlantis, Fisantekraal and Northern Works each consist of a single reactor unit, with all images stored in one folder. Cape Flats consists of four units (CF1–CF4) and Waterval of two (Top and Bottom), each with its own image folder. Every unit has its own aerobic zone and clarifier label file, except at Fisantekraal and Northern Works, where only clarifiers were labelled. Two of the 59 Atlantis images have no labels and are not part of the dataset.

Table 4.1 summarises the structure of the dataset across the five facilities.

Table 4.1: Number of units and labelled images per facility.

Facility	Units	Images
Atlantis	1	57
Cape Flats	4	382 (96+94+92+100)
Waterval	2	73 (37+36)
Fisantekraal	1	63
Northern Works	1	45
Total	9	620

4.4 Data Preparation

Before the labelled regions can be used to train a classifier, they need to be extracted from the full facility images and organised by class. This is handled by a preparation script (`prep.py`), which reads the VIA2 annotation files, crops out each labelled region, and saves the crops into a folder structure that can be loaded directly with `torchvision.datasets.ImageFolder`.

The script processes each facility one unit at a time and works through the aerobic zone and clarifier label files separately. For each labelled region, it converts the shape (a rectangle for an aerobic zone, a circle for a clarifier) to a bounding box, clamped to the image edges in case a region was drawn slightly outside the image. The class label is normalised to remove capitalisation inconsistencies and checked against the expected classes for that component. For a clarifier, every pixel outside the annotated circle is set to black, so that only the tank remains visible. The region is then cropped and saved as a JPG file in the folder matching its class label.

Each output filename encodes the facility, unit, and original image name, so that a crop can always be traced back to the source image it came from. For example, a crop from unit CF2 at Cape Flats would be saved with a filename beginning `CapeFlats_Images-CF2_...`, followed by the original image name and a region number.

The result is a folder structure with one top-level folder per component and one subfolder per class:

```
cnn_dataset/
    aerobic_zone/
        Dysfunctional/
        Functional/
        Suboptimal/
    clarifier/
        Dysfunctional/
        Empty/
        Functional/
        Scum/
```

The two classes that are excluded from the dataset (Section 4.5) have no folder. This

structure means the aerobic zone and clarifier sub-images are treated as two separate classification problems, each with its own set of class folders, rather than a single combined dataset.

4.5 Excluded from the dataset

Two annotated classes had too few examples to be used: a single aerobic zone labelled **Empty** and four clarifiers labelled **Stagnant**. Both were excluded, since so few examples cannot be split into training, development and test sets, and would not allow the CNN to learn a reliable representation of the class.

4.6 Data Splitting

In the following section the way the available data is split into training, development and testing partitions is described.

4.6.1 Leave-one-out Split

To evaluate how well the trained classifier generalises to WWTW facilities it has never encountered, the dataset was partitioned using a leave-one-out strategy rather than a random split. One facility, chosen from the five available (Atlantis, Cape Flats, Waterval, Fisantekraal, and Northern Works), was the test facility and withheld from training entirely: none of its images appear in the training or development sets, and it plays no part in training. Note that this is repeated once for each facility: five times for the clarifiers, and three times for the aerobic zones, which are only labelled at Atlantis, Cape Flats and Waterval. This guarantees that test performance reflects the model's ability to classify components at an unseen site.

This split was applied independently for each facility component, aerobic zone and clarifier, since the two have different classes and different image sets.

Once the test facility was removed, the remaining four facilities' images were divided into training and development sets. A random split at this stage risks an uneven mix: the development set could end up dominated by whichever facility or unit happened to contribute the most images, or by images from a narrow range of capture dates, which would give an unrepresentative estimate of development performance.

To avoid this, images were first grouped by facility, unit and class (some facilities have more than one unit, for example Cape Flats has 4 units each with its own set of images). Different units at the same facility can differ in lighting, or capture date, so treating them as separate groups during shuffling keeps that variation spread across both training and development rather than concentrated in one. Within each group, images were shuffled using a fixed random seed to mix capture dates before being partitioned according to a fixed development ratio (20%). Splitting within these smaller groups rather than across the whole set ensures that every facility, unit, and class combination among the four

training facilities is represented in both the training and development sets, in proportion to how much data it contributes. Figure A.1 in Appendix A shows the resulting split for the clarifiers when Waterval is withheld as the test facility.

4.6.2 Chronological Split

In addition to the leave-one-out approach described above, a chronological split was used to produce training, development, and test sets directly from the dataset, without holding any facility out. This split is made by capture date: every image captured from January 2020 onwards is a test image, and the images captured before January 2020 are divided into training and development sets in an 80/20 ratio, using the same facility, unit, class grouping and shuffling procedure described in the previous section. The split is made by date rather than at random for two reasons. First, the system would classify images that are newer than every image it was trained on. Second, a random split of the cropped images would put crops from the same satellite image, captured on the same date and in the same conditions, into both the training and the test set. The cut-off of January 2020 was chosen so that the test set of every facility contains every class that occurs at that facility. As a result, all 3 **Empty** clarifiers at Atlantis and all 7 **Dysfunctional** clarifiers at Waterval are in the test set. The resulting number of images per class is given in Section 5.7.

4.7 Relabelling

The dataset is new and was prepared shortly before the project started by annotators under the supervision of Prof F. de Wet. Early inspection found occasional labelling errors, so every clarifier image was reviewed manually with a relabelling notebook built on Jupyter widgets. The full set was reviewed, not just the images the classifier got wrong, because reviewing only those could introduce bias. The notebook shows each image with its current label and a button for each class. Corrections are written to a separate `relabel/` copy of the annotation files, so the original VIA2 labels stay intact.

Figure 4.6 shows a Cape Flats clarifier that was relabelled from **Scum** to **Empty**. The tank appears to have been drained: its floor is visible, and the dark band across it is the bridge's shadow, not scum. The review interface is shown in Figure A.2 in Appendix A.

The review proposed 293 corrections, and the supervisor checked each one. 166 were accepted, 99 were rejected, and 28 were given a different label, so 194 regions ended up with a label different from the original. A script applied these decisions to the `relabel/` copy. It first checked that each region still held the proposed label, and it logged every decision. These labels are used in every experiment in Chapter 5.

Table 4.2 gives the number of clarifier regions per class at each facility before (**Old**) and after (**New**) relabelling. The **Relabels** column gives the number of regions whose label changed.



Figure 4.6: A relabelled clarifier from Cape Flats, shown with some of its surroundings. The original label was "Scum", but the label was corrected to "Empty", since the tank appears to be drained, with its floor and the shadow of the bridge visible inside it.

Table 4.2: Clarifier regions per class, per facility, before (Old) and after (New) relabelling. Relabels is the number of regions whose label changed.

	Dysfunctional		**Empty**		**Functional**		**Scum**		
Facility	Old	New	Old	New	Old	New	Old	New	**Relabels**
Atlantis	0	0	2	3	44	44	64	63	2
Cape Flats	151	137	113	120	696	679	516	540	68
Waterval	78	7	0	0	95	89	111	188	80
Fisantekraal	19	17	9	9	106	110	118	116	8
Northern Works	127	131	13	13	113	81	152	180	36
TOTAL	**375**	**292**	**137**	**145**	**1054**	**1003**	**961**	**1087**	**194**

4.8 Final Dataset

Tables 4.3 and 4.4 give the final number of images per class, per unit, after relabelling. These are the counts used in every experiment. The aerobic zone dataset contains 1743 images, all from Atlantis, Cape Flats and Waterval, and the clarifier dataset contains 2527 images from all five facilities. Both datasets are imbalanced. Suboptimal is the smallest aerobic zone class with 223 images, against 813 for Dysfunctional. Among the clarifiers, Empty has only 145 images, against 1087 for Scum. The classes are also spread unevenly across facilities: Waterval Top accounts for 344 of the 707 Functional aerobic zones, and Northern Works accounts for 131 of the 292 Dysfunctional clarifiers. Section 5.4 describes how this imbalance is handled during training.

4.9 Conclusion

In this chapter, we described the five WWTW facilities, how their images were annotated, cropped and relabelled, the two ways the data is split, and the final number of images per class. In the next chapter, we describe the classification system and the experiments performed.

Table 4.3: Summary of image counts per class for the Aerobic Zones.

Facility	Dysfunctional	Functional	Suboptimal
Atlantis	22	72	19
Cape Flats 1	113	44	37
Cape Flats 2	146	21	21
Cape Flats 3	137	15	32
Cape Flats 4	166	12	22
Waterval Top	85	344	15
Waterval Bottom	144	199	77
TOTAL	**813**	**707**	**223**

Table 4.4: Summary of image counts per class for the Clarifiers.

Facility	Dysfunctional	Empty	Functional	Scum
Atlantis	0	3	44	63
Cape Flats 1	31	59	172	82
Cape Flats 2	58	35	176	95
Cape Flats 3	21	22	201	124
Cape Flats 4	27	4	130	239
Waterval Top	0	0	62	78
Waterval Bottom	7	0	27	110
Fisantekraal	17	9	110	116
Northern Works	131	13	81	180
TOTAL	**292**	**145**	**1003**	**1087**

Experimental Design

5.1 Overview

This chapter describes the classification system and the experiments run with it. The clarifiers and the aerobic zones are separate classification problems (Chapter 4) but use the same system, and every experiment is run for both unless stated otherwise. Sections 5.2 to 5.6 describe the shared parts of the system: the architectures and transfer learning, augmentation, class imbalance, hyperparameter tuning and the training loop. Three experiments are then run:

1. **Mixed-facility baseline** (Section 5.7): all five facilities are pooled and split chronologically. Images captured from January 2020 onwards are test images.
2. **Leave-one-facility-out** (Section 5.8): each facility is held out in turn as the test set. This gives five folds for the clarifiers and three for the aerobic zones, and estimates performance on an unseen facility.
3. **Binary (OK / Not-OK)** (Section 5.9): **Functional** is OK and all other classes are Not-OK.

Section 5.10 gives the evaluation metrics and Section 5.11 the Grad-CAM method. Figure 5.1 shows the pipeline. Each run, meaning one architecture trained for one component on one split, saves its test metrics, confusion matrix, training history and per-image predictions to a `results_*.pkl` file.

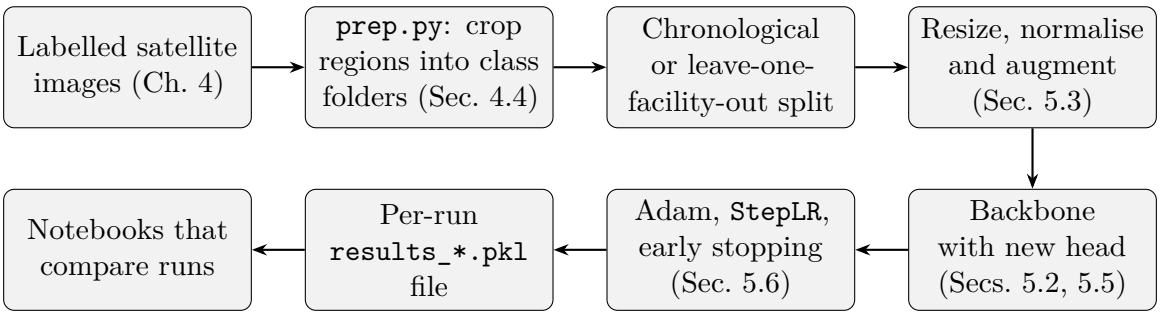


Figure 5.1: Experimental pipeline, from labelled satellite images to one results file per component, architecture and split.

5.2 Model Architectures and Transfer Learning

Four CNN architectures are compared in this project: ResNet-18 (Section 2.2.2), DenseNet-121 (Section 2.2.3), EfficientNet-B0 (Section 2.2.4) and ConvNeXt-Tiny (Section 2.2.5). ResNet-18 is trained twice, once fine-tuned from ImageNet weights and once from randomly initialised weights, which gives five models. Table 5.1 lists the sizes of the architectures, which range from 5.3 to 28.6 million parameters. All four are taken from the torchvision library [1], and the same five models are trained for both the clarifiers and the aerobic zones.

The ResNet-50 [5] and Inception-v3 [35] architectures were also included in this evaluation. However, neither had the highest macro F1 score for either component in either experiment, and due to space restrictions in this report, they are not reported on.

The ResNet-18 trained from randomly initialised weights is referred to as ResNet-18 (scratch). It is not tuned, and uses one fixed set of default hyperparameters (Section 5.5.3). It was added to the project to measure the effect of pretraining, that is, whether the pretrained architectures outperform the same architecture trained only on this project's own, small dataset.

Table 5.1: Parameter count of each of the four architectures evaluated, from smallest to largest, as defined in torchvision with the original 1000-class ImageNet classifier. The new classifier heads (Section 5.5) change these totals.

Architecture	Parameters
EfficientNet-B0	5.3M
DenseNet-121	8.0M
ResNet-18	11.7M
ConvNeXt-Tiny	28.6M

For each of the four pretrained architectures, transfer learning follows the same recipe. The architecture's convolutional backbone is loaded with weights pretrained on ImageNet [3, 4], specifically torchvision's `IMAGENET1K_V1` weights. Its original 1000-class ImageNet output layer is then discarded, and a new classifier head, sized by that architecture's own tuned hidden layers (H) and neurons (N) (Section 5.5), is put in its place: a stack of H `Linear+ReLU` pairs followed by a final `Linear` layer mapping to the project's number of classes. Every parameter in the network, backbone included, is then fine-tuned on the WWTW dataset.

As a design choice, ConvNeXt-Tiny's global average pooling layer is replaced with global max pooling, so the feature vector reaching its classifier head is composed of each channel's strongest activation rather than its average. This change is not required by the architecture, and the other architectures keep their default pooling.

Every image is scaled to fit the input size without changing its aspect ratio, and then padded with black to the full input size, with the image centred. The input size is 224×224 pixels for the clarifiers and 112×224 pixels for the aerobic zones. Every image is then normalised with the per-channel ImageNet mean and standard deviation since the pretrained backbones were trained on inputs normalised this way.

5.3 Augmentation

Augmentation is applied to the training images only, by PyTorch [36] each time an image is loaded, so every epoch sees a new random version of each image. The development and test images are not augmented. Three augmentations are applied, in this order:

- **Horizontal flip** with a probability of 50%.
- **Rotation** by a random angle between $-15°$ and $+15°$. The image keeps its size, so corners are cut off and the empty corners are filled with black.
- **Colour jitter**: brightness and contrast are multiplied by a random factor in $[0.6, 1.4]$, saturation by a factor in $[0.7, 1.3]$, and the hue is shifted by up to ± 0.1 of the colour wheel.

The images are taken from directly above, so a flipped or rotated clarifier or aerobic zone is still a realistic example of the same class. The black corners from rotation match the black mask that `prep.py` applies outside each clarifier and the padding added to the aerobic zones (Section 5.2). The colour jitter accounts for differences in lighting, sun glare and water colour between facilities and capture dates. Figure 5.2 shows the augmentations applied to one training image.

5.4 Class Imbalance

As shown in Tables 4.3 and 4.4 in Section 4.8, the classes of both components are highly imbalanced. For the clarifiers, **Functional** and **Scum** clarifiers make up most of the dataset (2090 of the 2527 images), while **Dysfunctional** and especially **Empty** clarifiers are much less common. For the aerobic zones, **Suboptimal** is the rare class, with only 223 of the 1743 images. If every image contributes equally to the loss, the network can reach a low loss by mostly assigning images to the common classes, and has little incentive to learn the rare ones.

To counter this, the cross-entropy loss is weighted per class, so that the network is penalised more for misclassifying an image from a rare class than from a common one. The weight of each class is inversely proportional to the number of training images in that class, and the weights are then normalised so that they sum to the number of classes. For K classes, where n_c is the number of training images in class c, the weight of class c is

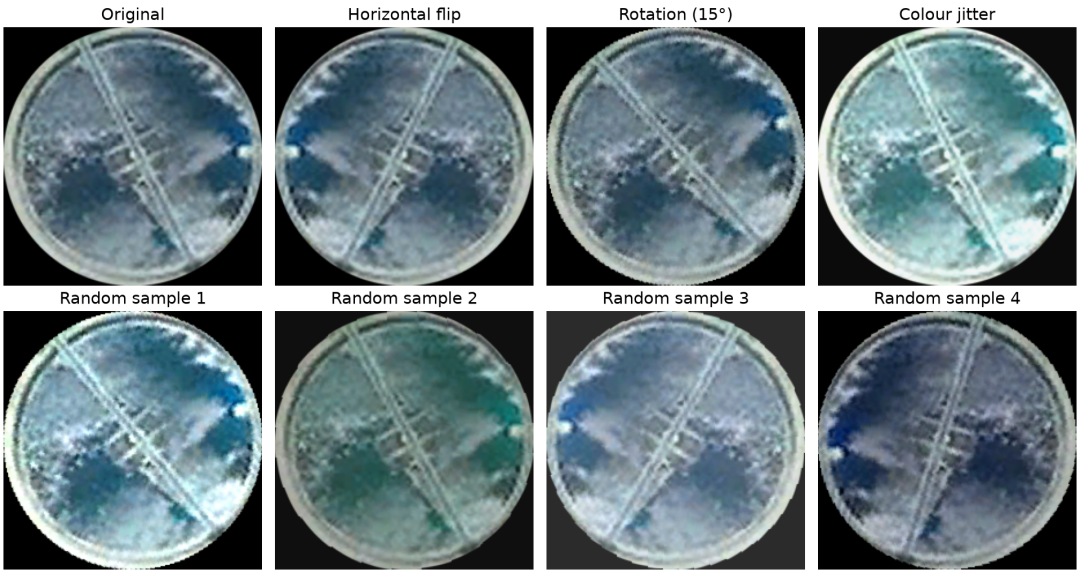


Figure 5.2: Augmentations applied to a single **Scum** clarifier from Cape Flats. Top row: the original image, followed by a horizontal flip, a rotation of 15°, and a random colour jitter, each applied on its own. Bottom row: four random outputs of the full augmentation pipeline (flip, rotation and colour jitter combined).

$$w_c = K \cdot \frac{1/n_c}{\sum_{j=1}^{K} 1/n_j}. \tag{5.1}$$

A class with average representation therefore gets a weight close to one, a rare class gets a weight larger than one, and a common class gets a weight smaller than one. These weights are passed to PyTorch's `CrossEntropyLoss`, which multiplies the loss of each image by the weight of its true class, and divides the sum over a batch by the sum of these weights rather than by the number of images. The training data itself is not resampled, so every training image is still used exactly once per epoch.

The weights are calculated from the training set only, and are recalculated for every split, since the class counts change depending on which facility is held out. Tables 5.2 and 5.3 list the number of training images per class and the resulting weights for every leave-one-facility-out split, and for the chronological split of Experiment 1.

The held-out facility can change the weights considerably. Cape Flats contains 120 of the 145 **Empty** clarifiers, so when it is held out, only 21 **Empty** images remain for training and the **Empty** weight rises to 3.09, about twenty times that of **Scum**. Waterval contains 543 of the 707 **Functional** aerobic zones, so when it is held out, **Functional** becomes a minority class in training (weight 1.19, compared with 0.34 to 0.68 in the other splits), while it dominates the test set.

The development loss uses the same weights, so the weighting also affects early stopping and checkpoint selection (Section 5.6).

Table 5.2: Number of training images per clarifier class and the resulting class weight w_c, for each held-out test facility and for the chronological split.

	Dysfunctional		**Empty**		**Functional**		**Scum**	
Test facility	Images	w_c	Images	w_c	Images	w_c	Images	w_c
Atlantis	236	1.10	116	2.24	769	0.34	822	0.32
Cape Flats	125	0.52	21	3.09	261	0.25	439	0.15
Fisantekraal	222	1.11	110	2.23	717	0.34	780	0.32
Northern Works	131	1.55	107	1.90	740	0.27	729	0.28
Waterval	230	1.12	118	2.18	733	0.35	722	0.36
None (chronological)	159	1.20	91	2.10	549	0.35	549	0.35

Table 5.3: Number of training images per aerobic zone class and the resulting class weight w_c, for each held-out test facility and for the chronological split.

	Dysfunctional		**Functional**		**Suboptimal**	
Test facility	Images	w_c	Images	w_c	Images	w_c
Atlantis	635	0.49	510	0.61	165	1.89
Cape Flats	202	0.82	493	0.34	90	1.84
Waterval	469	0.34	133	1.19	107	1.48
None (chronological)	454	0.53	357	0.68	136	1.79

5.5 Hyperparameter Tuning

Some hyperparameters were fixed for every architecture: Adam's β_2 (0.999, the default from Adam's original paper [37]), the L_2 weight decay (10^{-4}), a maximum of 100 epochs, an early-stopping patience of 10 epochs and a random seed of 42. The remaining hyperparameters, namely the number of hidden layers H and neurons per layer N of the classifier head, the learning rate, Adam's β_1, the `StepLR` step size and the batch size, were tuned with Optuna [38], a hyperparameter optimisation library. This was done for the four pretrained models, separately for the clarifiers and the aerobic zones, giving eight searches. ResNet-18 (scratch) was not tuned (Section 5.5.3).

5.5.1 Why Optuna

The six tuned hyperparameters interact, so tuning them one at a time can miss good combinations, and a grid search over all six is too expensive. Optuna searches all six jointly with a Tree-structured Parzen Estimator (TPE) sampler [39], which samples new candidates from the regions of the search space where earlier trials scored well. A `MedianPruner` also stops a trial early if its development accuracy falls below the median of the earlier trials at the same epoch.

5.5.2 Search Procedure

The first ten trials of every search are sampled at random, and the last ten are guided by TPE. The pruner only becomes active after five trials have completed, and does not prune a trial before its fifth epoch. Every trial uses the same training loop as the final models (Section 5.6) and is scored by its development accuracy at the epoch with the lowest development loss, which is the checkpoint that is kept when a model is trained.

To limit the compute cost, each search was run once rather than on every fold: on the leave-one-facility-out fold with Northern Works held out for the clarifiers, and with Atlantis held out for the aerobic zones, since Northern Works has no aerobic zones. The selected values are used for every fold and for the chronological split (Sections 5.7 and 5.8).

Table 5.4 gives the search range of each hyperparameter and the values selected for each architecture. All 160 trials are listed in Appendix B.

Table 5.4: Search range of each tuned hyperparameter, and the values used for each architecture and component, with the development accuracy of the selected trial. The learning rate is sampled log-uniformly, and the batch size from $\{8, 16, 24, 32, 48, 64, 96, 128\}$. H=0 means the classifier head is a single linear layer, and N is unused. The ResNet-18 (scratch) values were not tuned (Section 5.5.3).

Architecture	H	N	**LR**	β_1	**Step**	**Batch**	**Dev. acc.**
Search range	$\{0, \dots, 3\}$	$\{512, 1024, 2048\}$	$[10^{-6}, 10^{-2}]$	$[0.5, 0.999]$	$[5, 40]$	$\{8, \dots, 128\}$	–
Clarifiers							
ResNet-18	1	512	2.24×10^{-5}	0.971	16	32	85.78%
DenseNet-121	2	2048	1.38×10^{-4}	0.731	24	24	86.75%
EfficientNet-B0	0	–	8.76×10^{-4}	0.671	30	96	88.43%
ConvNeXt-Tiny	1	1024	9.93×10^{-5}	0.855	24	32	90.12%
ResNet-18 (scratch)	3	1024	1×10^{-4}	0.9	15	32	–
Aerobic zones							
ResNet-18	0	–	8.42×10^{-5}	0.794	17	24	86.56%
DenseNet-121	0	–	1.39×10^{-4}	0.678	27	32	88.75%
EfficientNet-B0	2	2048	1.72×10^{-4}	0.750	22	16	88.12%
ConvNeXt-Tiny	2	2048	6.78×10^{-5}	0.512	27	8	91.56%
ResNet-18 (scratch)	3	1024	1×10^{-4}	0.9	15	32	–

5.5.3 ResNet-18 (Scratch)

ResNet-18 (scratch) is trained from randomly initialised weights rather than fine-tuned (Section 5.2). It was not included in the search, and uses the same default values for both components (Table 5.4): three hidden layers of 1024 neurons, a learning rate of 10^{-4}, Adam's default β_1 of 0.9, a step size of 15 and a batch size of 32. It is trained with the same loop and early stopping as the other models (Section 5.6). Since its hyperparameters also differ from those of the pretrained ResNet-18, the difference between the two models includes the effect of tuning as well as that of pretraining.

5.6 Training Procedure

Every architecture is trained with the same procedure.

Rather than using one fixed learning rate for an entire run, every architecture uses `StepLR` to cut the learning rate by a factor of ten every time a fixed, architecture-specific number of epochs (the step size) has passed (Section 5.5), since large steps are useful early on to make fast progress but become counter-productive once the model is close to a good solution, where they cause it to overshoot instead of converging.

Training is allowed to run for up to 100 epochs, but is stopped early if development loss has not improved for 10 consecutive epochs, since the point at which a given architecture stops improving and starts overfitting is not known ahead of time and differs from one architecture to the next. Development *loss*, rather than accuracy, is used to decide both when to stop and which epoch's weights to keep: whichever epoch achieved the lowest development loss is the one whose weights are restored and carried forward to testing, rather than simply whatever weights training happened to end on. Figure 5.3 summarises this loop.

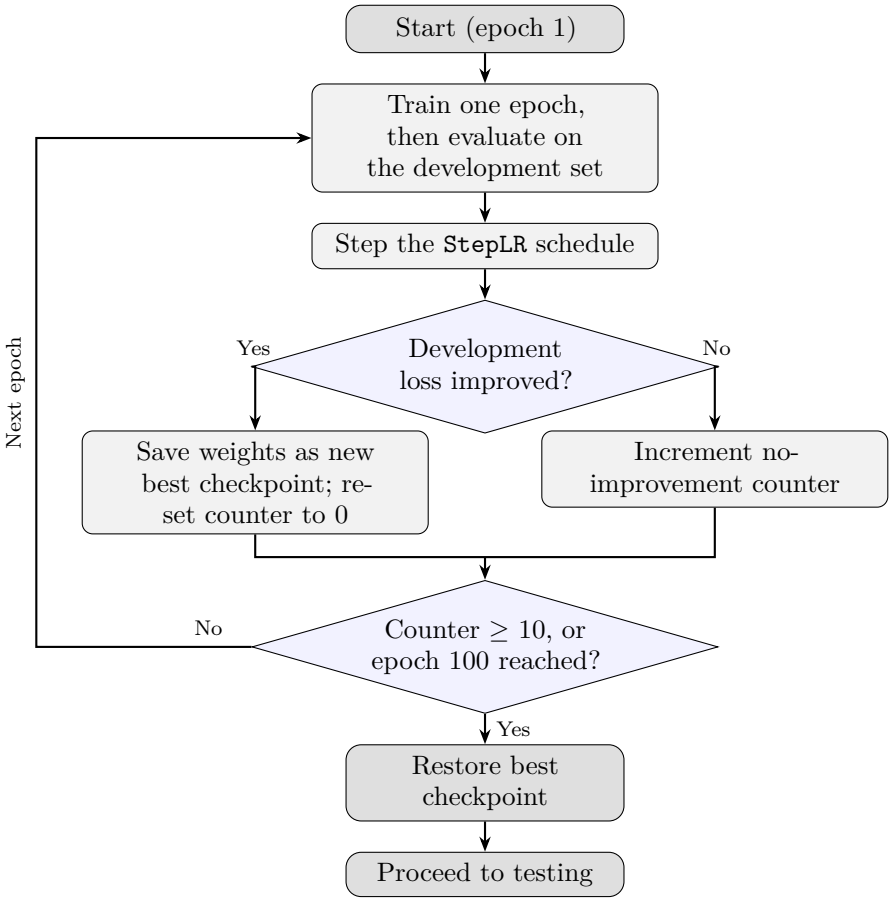


Figure 5.3: The training loop used for every architecture: an epoch of training and development-set evaluation, a scheduler step, a checkpoint decision based on development loss, and a stopping check repeated every epoch until training ends.

The same fixed seed used throughout this project (Section 5.5) also covers the random initialisation of each architecture's new classifier head and the order in which the training images are shuffled, in addition to the dataset splitting described in Chapter 4. A repeated run therefore starts from the same weights and splits.

5.7 Experiment 1: Mixed-Facility Baseline

The first experiment does not take into account which facility an image comes from. The images of all five facilities are pooled and divided into training, development and test sets using the chronological split described in Chapter 4. Every image captured from January 2020 onwards is a test image. The images captured before January 2020 are grouped by facility, unit and class, each group is shuffled using the fixed random seed of 42, and the group is then divided in an 80/20 ratio into training and development images. A group with at least three images always receives at least one development image. Table 5.5 gives the resulting number of images per class.

Table 5.5: Number of images per class in the training, development and test sets of the chronological split used in Experiment 1. The test set contains every image captured from January 2020 onwards.

Component	Class	Train	Development	Test	Total
Clarifier	Dysfunctional	159	37	96	292
	Empty	91	20	34	145
	Functional	549	133	321	1003
	Scum	549	133	405	1087
	Total	1348	323	856	2527
Aerobic zone	Dysfunctional	454	110	249	813
	Functional	357	86	264	707
	Suboptimal	136	33	54	223
	Total	947	229	567	1743

All five models are trained using this division of the data, for both the clarifiers and the aerobic zones, with the hyperparameter values in Table 5.4 (Section 5.5). These were tuned on a leave-one-facility-out fold, so some of the test images of Experiment 1 were part of the development set on which the hyperparameters were chosen, although they were never used to train the models that are tested on them.

Experiment 1 is also used to measure the effect of augmentation. Every model is trained a second time on the same data but without augmentation (resizing and normalisation is maintained for all train, development and test images). The data, hyperparameters, class weights and random seed are the same in both experiments, to ensure that augmentation is the only difference. Experiment 1 therefore consists of 20 training runs: five models, for

two components, each trained with and without augmentation. The results are given in Appendix C.3.

The purpose of this experiment is to estimate how well the models perform at facilities whose earlier images are part of the training data. This is the usual situation in the intended use of the system (Chapter 1): new images of a facility are classified, an operator relabels the flagged images, and the relabelled images are added to the training data. The test images were all captured after the training and development images, as they would be in use. Since a satellite image has a single capture date, no test image comes from the same satellite image as a training or development image.

A facility that is added to the system has no labelled images until its first images have been reviewed. A model could also learn cues that are specific to one facility, such as the colour of the water at that facility, and still perform well on this split, since the same cues are present in the test set. The results of Experiment 1 therefore do not show how a model would perform at a facility it has never seen, which is the reason for Experiment 2. The difference between the two experiments gives an indication of how much performance is lost when a model is applied to a facility it was not trained on.

5.8 Experiment 2: Leave-One-Facility-Out Evaluation

Experiment 2 uses the leave-one-facility-out split described in Chapter 4 and illustrated in Figure A.1. One facility is held out as the test set, and the images of the remaining facilities are divided into training and development sets in an 80/20 ratio, grouped by facility, unit and class in the same way as in Experiment 1. This is repeated until every facility has been held out once, and every repetition is referred to as a fold. The images of the test facility are never used to update the weights of the network, to calculate the class weights, to decide when training stops or to select the checkpoint that is kept. The test results of a fold are therefore an estimate of how a model would perform at a facility it has never seen.

Every architecture is trained separately for every fold, using the hyperparameters (Section 5.5), class-weighted loss (Section 5.4), augmentation (Section 5.3) and training loop (Section 5.6) described earlier in this chapter, with the same hyperparameters used throughout. For the clarifiers, this gives five folds and 25 training runs (one for each model and fold). Aerobic zones are only labelled at Atlantis, Cape Flats and Waterval (Chapter 4), so in this case there are only three aerobic zone folds and 15 training runs. Since Fisantekraal and Northern Works contribute no aerobic zone images, every aerobic zone model is trained on the two aerobic facilities that are not held out. Table 5.6 gives the size of every fold, together with the number of test images per class.

In this experiment the folds differ considerably in size. Cape Flats contains 1476 of the 2527 clarifier images, so when it is held out, the model is trained on only 846 images, less than half of what the other folds are trained on, and tested on the largest test set.

Table 5.6: Number of training, development and test images in each leave-one-facility-out fold, and the number of test images per class. The test set is the held-out facility, and the training and development sets are drawn from the remaining facilities.

				Test images per class			
Test facility	**Train**	**Dev.**	**Test**				
Clarifiers				Dysfunctional	Empty	Functional	Scum
Atlantis	1943	474	110	0	3	44	63
Cape Flats	846	205	1476	137	120	679	540
Fisantekraal	1829	446	252	17	9	110	116
Northern Works	1707	415	405	131	13	81	180
Waterval	1803	440	284	7	0	89	188
Aerobic zones				Dysfunctional	Functional	Suboptimal	
Atlantis	1310	320	113	22	72	19	
Cape Flats	785	192	766	562	92	112	
Waterval	709	170	864	229	543	92	

Waterval plays a similar role for the aerobic zones, with 864 of the 1743 images. The class composition of the test sets also differs between facilities. The Atlantis test set contains no **Dysfunctional** clarifiers and only three **Empty** clarifiers, and the Waterval test set contains no **Empty** clarifiers and only seven **Dysfunctional** clarifiers, while **Dysfunctional** clarifiers make up almost a third of the Northern Works test set. The recall of a class cannot be calculated at a facility where that class does not occur, and a class with only a handful of test images at a facility leads to an uncertain estimate. For the same reason, the accuracy of one fold cannot be compared directly with that of another: a model that recognises one class well and another poorly will score very differently depending on which class dominates the test set. Macro-averaged metrics, which give every class the same weight, are averaged only over the classes that occur in the test set of a fold. Otherwise, a class without any test images would count as a score of zero, and would incorrectly lower the macro average of the Atlantis and Waterval clarifier tets sets.

To summarise the performance of an architecture, the results per test fold are averaged, with every fold counting equally regardless of its size. The standard deviation across test folds is reported alongside the mean. Pooling the test images of all folds (micro-averaging) would let Cape Flats determine most of the clarifier result. The standard deviation shows how much the performance of an architecture depends on the facility it is tested on.

One part of this experiment is not fully independent of the test facility. The hyperparameters were tuned once per architecture, with Northern Works held out for the clarifiers and Atlantis held out for the aerobic zones (Section 5.5), and the development set of that fold contains images from the other facilities. For the other folds, images from the test facility were therefore part of the development set on which the hyperparameters were

chosen, although they were never used to train the model that is tested on them. Since only six hyperparameters were chosen in this way, the effect is likely to be small, but only the Northern Works clarifier fold and the Atlantis aerobic zone fold are fully independent of the tuning.

5.9 Experiment 3: Binary (OK / Not-OK) Reformulation

The multi-class models distinguish between several ways in which a component can fail, but for an operator the most important question is whether a component needs attention or not. A **Scum** clarifier classified as **Dysfunctional** is still flagged for inspection, whereas a **Scum** clarifier classified as **Functional** is missed completely. Experiment 3 therefore evaluates the models as binary classifiers, which only decide whether a component is functioning correctly (OK) or not (Not-OK). For the clarifiers, **Functional** means OK, while **Dysfunctional**, **Empty** and **Scum** are Not-OK. For the aerobic zones, **Functional** means OK, while **Suboptimal** and **Dysfunctional** are Not-OK.

No separate binary models are trained. Instead, the binary decision is derived from the multi-class models of Experiments 1 and 2. The Not-OK score of an image is the total probability that the model assigns to the Not-OK classes,

$$s = 1 - P(\text{Functional}), \tag{5.2}$$

and the image is classified as Not-OK when $s \geq t$, for a threshold t. The binary results therefore come from exactly the same models as the multi-class results, so the two can be compared directly. The binary evaluation is carried out on both the chronological split of Experiment 1 and the leave-one-facility-out split of Experiment 2.

With Not-OK treated as the positive class, the recall (or sensitivity) is the fraction of Not-OK components that are flagged as such, while the specificity is the fraction of OK components that are correctly left unflagged. These two kinds of mistake do not have the same cost. A false alarm only costs an operator the time to look at the image and make his or her own confirmatory assessment, while a missed Not-OK component may go unnoticed until the next satellite image of the facility becomes available. The decision threshold therefore must be chosen so that the recall of the Not-OK class reaches a target of 95% on the development set of the same run. The recall is evaluated at 500 evenly spaced thresholds between 0 and 1, and the largest threshold that still reaches the target is selected, since this is the threshold that also raises the fewest false alarms. The recall, specificity and precision are then calculated on the test set at this threshold. For the leave-one-facility-out models of Experiment 2, every fold has its own threshold, chosen on the development set of that fold, which contains no images of the held-out facility.

The ROC curve and its AUC are also calculated for the binary problem, since they describe the trade-off between recall and specificity over all thresholds rather than at a

single one. For the leave-one-facility-out models, the binary results are summarised across held-out facilities in the same way as the multi-class results (Section 5.8). The ROC curve of every fold is interpolated onto a common set of false positive rates before the curves are averaged, and the binary confusion matrix at the chosen threshold is normalised per row, so that it shows rates rather than counts, before it is averaged.

Since the test set plays no part in choosing the threshold, the test recall can be lower or higher than 95%. The development set is also used to select the checkpoint (Section 5.6), so the development recall may be slightly optimistic. For the leave-one-facility-out split, the test recall is also reported for every held-out facility separately. In practice, the threshold would have to be fixed before the system is used at a new facility, so a test recall well below 95% at some facilities would mean that a threshold chosen on known facilities cannot be relied on to reach the target recall at a new one.

5.10 Evaluation Metrics

Every run is evaluated once on its test set, after training has finished and the checkpoint with the lowest development loss has been restored (Section 5.6). A test image is assigned to the class with the highest softmax probability, and the metrics described in Section 2.3 are calculated from these classifications and probabilities.

Of these metrics, recall is the most important in the context of this project. The system is meant to flag components that are not working properly, so that an operator can inspect them. A component with low recall for a failure class, such as a **Dysfunctional** clarifier or a **Suboptimal** aerobic zone, would be missed and could go unnoticed until the next satellite image of the facility, whereas a false positive only costs an operator the time to look at an image. For this reason, the recall of the failure classes is reported alongside the macro-averaged scores, and the operating point in Experiment 3 is chosen to reach a target recall rather than the highest accuracy.

5.11 Qualitative Analysis: Grad-CAM

Classification performance alone does not show what in an image a model bases its decision on, so a model could rely on something unrelated to the state of the component, such as the water colour at a particular facility or the rim of a clarifier. Grad-CAM (Section 2.4) is used to check this, by highlighting the regions of an image that contributed the most to a classification.

5.11.1 Method

Grad-CAM was applied to the output of the last feature block of each selected model, using the `pytorch-grad-cam` library [40]. The heatmaps are superimposed on the input images, where red regions contributed the most to the classification. Every heatmap is normalised separately, so the colours cannot be compared between images.

5.11.2 Models and Example Selection

For each component, the architecture with the highest mean macro-averaged F1 test score in the leave-one-facility-out experiment (Section 5.8) was selected for analysis using one of the held-out test sets. For the clarifiers, Fisantekraal was used, since it contains all four clarifier classes, and for the aerobic zones, Waterval was used, since it has the most aerobic zone images. For each class, one correctly classified image was chosen at random, along with one image from each of the most common mistakes. The heatmaps are calculated for the assigned class, so for a mistake they show what the model took as evidence to arrive at its incorrect decision.

Results

6.1 Overview

The results are presented for two situations. In the first, which is the usual situation in the intended use of the system, earlier images of a facility have been relabelled and are part of the training data. Experiment 1 estimates the performance in this situation, since all of its test images were taken after its training images (Section 5.7). In the second, which occurs when a facility is added, none of its images have been labelled. Experiment 2 estimates the performance in this situation.

Section 6.2 gives the multi-class results, Section 6.3 the binary OK / Not-OK results at a threshold chosen for a Not-OK recall of 95% on the development set, and Section 6.4 the Grad-CAM heatmaps of the two selected models. The full results are given in Appendix C. In every table, the best value in every column is shown in bold.

6.2 Multi-Class Classification

Table 6.1: Macro-averaged F1 score and recall of every architecture on the chronological split (Exp. 1) and averaged across held-out facilities (Exp. 2, mean ± standard deviation). Δ is the change in the F1 score from Experiment 1 to Experiment 2, in percentage points.

	Macro F1			**Macro recall**	
Architecture	Exp. 1	Exp. 2	Δ	Exp. 1	Exp. 2
Clarifiers					
ResNet-18	0.732	0.668 ± 0.068	-6.4	0.733	0.648
EfficientNet-B0	0.691	0.758 ± 0.098	+6.7	0.715	**0.749**
DenseNet-121	**0.772**	0.724 ± 0.086	-4.8	**0.760**	0.722
ConvNeXt-Tiny	0.748	**0.775** ± 0.062	+2.7	0.743	0.748
ResNet-18 (scratch)	0.667	0.662 ± 0.064	-0.5	0.704	0.666
Aerobic zones					
ResNet-18	0.638	0.435 ± 0.081	-20.3	0.639	0.517
EfficientNet-B0	**0.740**	0.413 ± 0.130	-32.7	**0.733**	0.484
DenseNet-121	0.662	**0.495** ± 0.033	-16.7	0.664	**0.541**
ConvNeXt-Tiny	0.724	0.451 ± 0.194	-27.3	0.724	0.537
ResNet-18 (scratch)	0.624	0.331 ± 0.101	-29.3	0.624	0.399

Table 6.1 shows that the macro F1 score of the aerobic zones decreases from Experiment 1 to Experiment 2 for every architecture, by between 16.7 and 32.7 percentage points. For the clarifiers, the change ranges from a decrease of 6.4 to an increase of 6.7 percentage points, so the clarifier models perform about as well at a new facility as on new images of a known facility. The aerobic zone models therefore generalise worse to a new facility

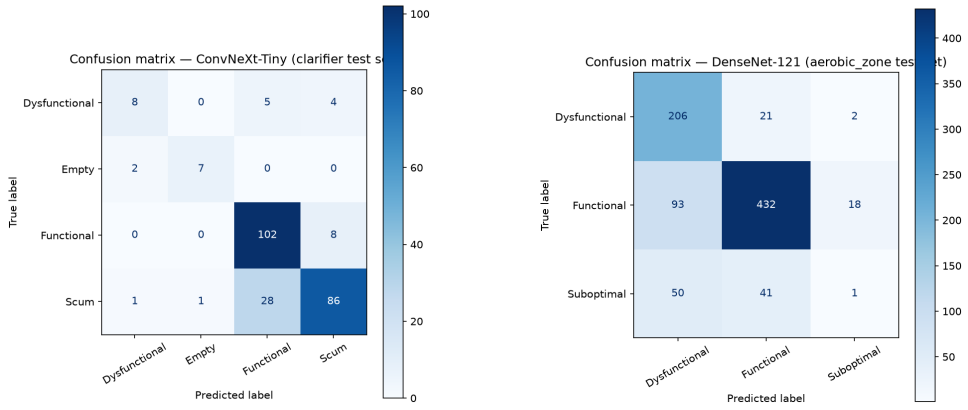
than the clarifier models. In Experiment 1, the **Empty** class has the lowest clarifier F1 score of every architecture (Appendix C.1). DenseNet-121 has the highest clarifier F1 score in Experiment 1 and ConvNeXt-Tiny in Experiment 2, while EfficientNet-B0 has the highest aerobic zone F1 score in Experiment 1 and DenseNet-121 in Experiment 2. In Experiment 2, the standard deviation of the clarifier F1 score across facilities (0.062 to 0.098) is of the same size as the differences between the pretrained architectures (0.668 to 0.775), so the test facility has about as large an effect on the result as the architecture. ResNet-18 trained from scratch has the lowest macro F1 score for both components in both experiments. Compared with the pretrained ResNet-18, its clarifier F1 score is 6.5 and 0.6 percentage points lower in Experiments 1 and 2, and its aerobic zone F1 score is 1.4 and 10.4 percentage points lower. Part of this difference is due to tuning (Section 5.5.3), but since pretraining gave a higher score in every case, it is worth using.

Table 6.2: Recall of every failure class on the chronological split (Exp. 1) and averaged across the held-out facilities at which the class occurs (Exp. 2).

	Clarifiers			**Aerobic zones**	
Architecture	Dysfunctional	Empty	Scum	Dysfunctional	Suboptimal
Experiment 1					
ResNet-18	0.792	0.588	0.827	0.932	0.167
EfficientNet-B0	0.750	0.500	0.746	0.884	**0.389**
DenseNet-121	0.844	**0.647**	**0.859**	**0.940**	0.241
ConvNeXt-Tiny	**0.885**	0.382	0.852	0.916	0.370
ResNet-18 (scratch)	0.823	0.559	0.607	0.811	0.185
Experiment 2					
ResNet-18	0.623	0.461	0.756	**0.726**	0.072
EfficientNet-B0	0.661	**0.662**	0.802	0.458	0.198
DenseNet-121	0.728	0.560	0.763	0.712	0.184
ConvNeXt-Tiny	0.710	0.505	**0.833**	0.593	0.224
ResNet-18 (scratch)	**0.772**	0.361	0.717	0.244	**0.294**

Table 6.2 shows that the largest decreases in recall from Experiment 1 to Experiment 2 occur for the **Dysfunctional** aerobic zones. The recall of the **Suboptimal** class is at most 0.389 in Experiment 1 and at most 0.294 in Experiment 2, so most **Suboptimal** aerobic zones are not identified, even at a known facility.

Figure 6.1 shows the most common mistakes of both selected models. At Fisantekraal, 28 of the 116 **Scum** clarifiers are classified as **Functional**. At Waterval, 93 of the 543 **Functional** aerobic zones are classified as **Dysfunctional**, and 50 and 41 of the 92 **Suboptimal** zones as **Dysfunctional** and **Functional** respectively. A failure class classified as **Functional** is a component that would not be flagged for inspection.



(a) Clarifiers, ConvNeXt-Tiny, Fisantekraal held out

(b) Aerobic zones, DenseNet-121, Waterval held out

Figure 6.1: Confusion matrices of the architecture with the highest mean macro F1 score in Experiment 2 for each component, on the held-out facility used for the Grad-CAM analysis (Section 6.4). The confusion matrices of the other folds are given in Appendix C.2.

6.3 Binary (OK / Not-OK) Classification

Table 6.3: Binary OK / Not-OK results of every architecture on the chronological split (Exp. 1) and averaged across held-out facilities (Exp. 2). The AUC does not depend on a threshold. The recall, specificity and precision of the Not-OK class are given on the test set, at the threshold chosen for a Not-OK recall of 95% on the development set (Section 5.9). The thresholds are listed in Appendix C.4.

	AUC		**Recall**		**Specificity**		**Precision**	
Architecture	Exp. 1	Exp. 2	Exp. 1	Exp. 2	Exp. 1	Exp. 2	Exp. 1	Exp. 2
Clarifiers								
ResNet-18	0.914	0.892 ± 0.025	**0.974**	0.867	0.502	0.540	0.765	0.806
EfficientNet-B0	0.941	0.938 ± 0.039	0.961	0.902	0.595	0.702	0.798	0.858
DenseNet-121	0.901	0.926 ± 0.029	0.959	0.897	0.502	0.710	0.762	0.857
ConvNeXt-Tiny	**0.949**	**0.953** ± 0.024	0.968	0.886	**0.651**	**0.846**	**0.822**	**0.914**
ResNet-18 (scratch)	0.893	0.893 ± 0.077	0.964	**0.916**	0.464	0.508	0.750	0.795
Aerobic zones								
ResNet-18	0.959	0.814 ± 0.048	0.931	0.846	0.826	0.573	0.860	0.664
EfficientNet-B0	0.966	0.819 ± 0.020	0.941	0.740	0.795	0.701	0.841	0.748
DenseNet-121	0.959	0.846 ± 0.040	0.924	**0.864**	0.830	0.602	0.862	0.706
ConvNeXt-Tiny	**0.976**	**0.866** ± 0.029	**0.950**	0.692	0.833	**0.831**	0.867	**0.780**
ResNet-18 (scratch)	0.949	0.697 ± 0.139	0.881	0.719	**0.856**	0.558	**0.875**	0.607

Table 6.3 gives the binary results at the threshold chosen on the development set, in Experiment 2 as an average across facilities (Section 5.9). The specificity is the fraction of OK components that are not flagged. In Experiment 1, the test recall is 0.959 to 0.974 for the clarifiers, so the target of 95% is reached by every architecture, and 0.881 to 0.950 for the aerobic zones, so it is missed by four of the five. In Experiment 2, the mean test recall is 0.867 to 0.916 for the clarifiers and 0.692 to 0.864 for the aerobic zones, and no

architecture reaches the target. The clarifier specificity is higher in Experiment 2 than in Experiment 1 for every architecture, but this goes together with the lower recall: fewer components are flagged, both Not-OK and OK.

Table 6.4: Not-OK recall and specificity for each held-out facility at the threshold chosen on the development set of that fold, for the ConvNeXt-Tiny clarifier model and the DenseNet-121 aerobic zone model. The results of every architecture are given in Appendix C.4.

	Clarifiers		**Aerobic zones**	
Held-out facility	Recall	Specificity	Recall	Specificity
Atlantis	0.758	1.000	0.878	0.444
Cape Flats	0.956	0.704	0.935	0.554
Fisantekraal	0.810	0.882	–	–
Northern Works	0.985	0.790	–	–
Waterval	0.923	0.854	0.779	0.808

Table 6.4 shows that at individual facilities, the clarifier recall ranges from 0.758 to 0.985 and the aerobic zone recall from 0.779 to 0.935. The target of 95% is reached at two of the five clarifier facilities and at none of the aerobic zone facilities. Over all architectures, the lowest recall at a single facility is 0.667 for the clarifiers and 0.447 for the aerobic zones (Appendix C.4). A threshold chosen on the development set of other facilities therefore does not reach the target recall at a new facility. For a facility whose relabelled images are part of the training data, the clarifier threshold chosen on the development set did reach the target in Experiment 1, but the aerobic zone threshold did not for most architectures.

6.4 Qualitative Analysis: Grad-CAM

The Grad-CAM method and example selection described in Section 5.11 were applied to the ConvNeXt-Tiny clarifier model, tested on Fisantekraal, and the DenseNet-121 aerobic zone model, tested on Waterval (Figure 6.1). The last feature blocks of these networks have a spatial size of 7×7 and 3×7 respectively, so the heatmaps are coarse.

6.4.1 Clarifiers

Figure 6.2 shows the clarifier examples. For the correctly classified **Dysfunctional** and **Scum** clarifiers, the highest weights lie on the surface inside the tank: on the green surface, and on the uneven patches near the centre, respectively. For the **Empty** clarifier, the highest weight lies on the shadow at the top of the tank and on the rim and background next to it. For the **Functional** clarifier, the highest weights lie on the rim and the background at the top and bottom of the tank, where the bridge meets the rim, with low weights on the black surface.

Most of the ConvNeXt-Tiny heatmaps highlight one or two small regions rather than a large part of the surface. A possible cause is the global max pooling of ConvNeXt-Tiny (Section 5.2), which passes only the largest activation of each feature map to the classifier head.

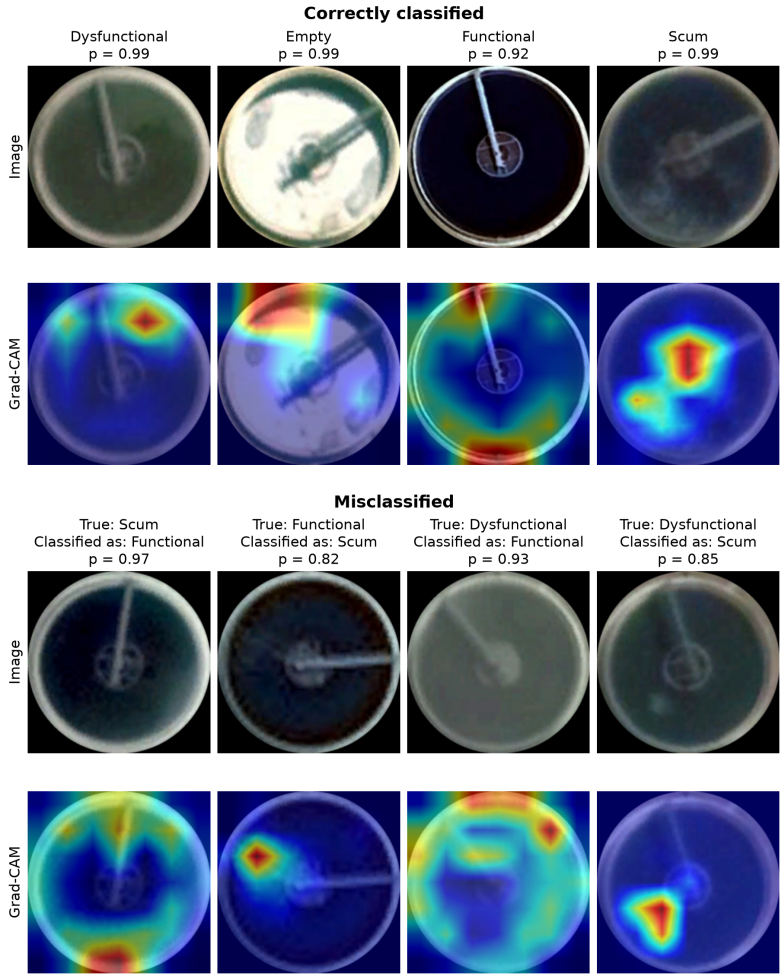


Figure 6.2: Grad-CAM heatmaps of the ConvNeXt-Tiny clarifier model, for test images from Fisantekraal, which was held out during training. Top: one correctly classified clarifier per class. Bottom: one clarifier from each of the four most common mistakes. Every heatmap is calculated for the assigned class, and p is the softmax probability of that class. Red regions contributed the most to the classification.

In the **Scum** clarifier classified as **Functional** ($p = 0.97$), the lighter surface around the centre of the tank receives little weight, and the highest weights lie on the rim at the top and bottom of the tank, as for the correctly classified **Functional** clarifier. The class requires the whole surface to be black, but the rim does not show this. In the **Functional** clarifier classified as **Scum**, the highest weight lies on a small region of the surface near the left edge of the tank, which is slightly lighter than the rest of the surface.

Of the 17 **Dysfunctional** clarifiers at Fisantekraal, 9 were misclassified. For the clarifier classified as **Functional**, the weight is spread over the rim and the surface, with

the highest weight on the rim. The surface of this clarifier is pale grey rather than the darker green of the correctly classified example. For the clarifier classified as **Scum**, the highest weight lies on a dark spot on the surface below the centre of the tank.

6.4.2 Aerobic Zones

Figure 6.3 shows the aerobic zone examples. For the correctly classified **Functional** zone ($p = 0.80$), the heatmap covers the centre of the zone, where the leopard print pattern is visible. For the correctly classified **Dysfunctional** zone ($p = 0.84$), it lies on the smooth surface in the right half of the zone. The correctly classified **Suboptimal** zone has $p = 0.40$, slightly above the 0.33 of a random guess between three classes, and its heatmap lies on a darker band at the lower right of the zone. In several heatmaps, part of the weight lies on the black bars added by the padding (Section 5.2). The last feature block has only three rows, so the bottom row covers both the bar and the lower edge of the zone, and these heatmaps do not show whether the model uses the bars.

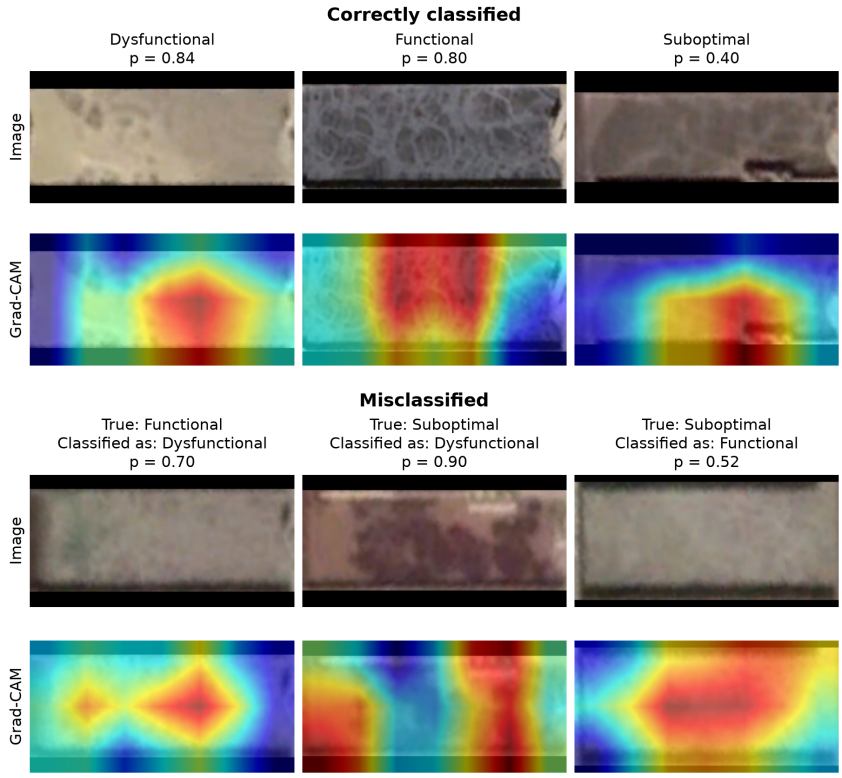


Figure 6.3: Grad-CAM heatmaps of the DenseNet-121 aerobic zone model, for test images from Waterval, which was held out during training. Top: one correctly classified aerobic zone per class. Bottom: one aerobic zone from each of the three most common mistakes. Every heatmap is calculated for the assigned class, and p is the softmax probability of that class. Red regions contributed the most to the classification.

The **Functional** zone classified as **Dysfunctional** ($p = 0.70$) is blurred, and no leopard print pattern is visible on its surface. Its heatmap lies on the centre of the zone. In the

Suboptimal zone classified as **Dysfunctional** ($p = 0.90$), the heatmap lies on the left end of the zone and on a band in the right half, where the surface shows dark patches. The **Suboptimal** zone classified as **Functional** ($p = 0.52$) has a plain grey surface, and its heatmap is spread over the centre of the zone. A zone is **Suboptimal** when a third or more of its surface shows no leopard print (Chapter 4), and in neither **Suboptimal** example is a pattern clearly visible. The aerobic zone labels were not reviewed (Chapter 4), so some of these mistakes may be caused by the labels.

6.4.3 Summary of Observations

For the correctly classified **Dysfunctional** and **Scum** clarifiers and **Functional** and **Dysfunctional** aerobic zones, the heatmaps lie on the features that the labels are based on. For the **Functional** and **Empty** clarifiers, they lie largely on the rim and the background, which do not indicate the state of the clarifier. The most common clarifier mistake, a **Scum** clarifier classified as **Functional**, is also based on the rim. When the surface shows little evidence, as in the blurred or plain aerobic zones, the heatmaps spread over the zone. These observations are based on a few examples.

6.5 Summary of Findings

In this chapter we have presented the multi-class and binary results of the two experiments, and the Grad-CAM heatmaps of the two selected models. At a known facility, the best clarifier and aerobic zone models reach a similar macro F1 score. At a new facility, the clarifier models perform about as well as at a known facility, while the aerobic zone models perform considerably worse. The differences between the pretrained architectures are of the same size as the differences between the held-out facilities, and ResNet-18 trained from scratch performs worst in both experiments. Most **Suboptimal** aerobic zones are not identified, even at a known facility.

When the threshold is set so that 95% of the Not-OK components in the development set are flagged, the test recall at a known facility is 0.959 to 0.974 for the clarifiers and 0.881 to 0.950 for the aerobic zones, and about a third of the OK clarifiers and 15% of the OK aerobic zones are also flagged by the best models. At a new facility, the mean test recall is 0.867 to 0.916 for the clarifiers and 0.692 to 0.864 for the aerobic zones, so a threshold chosen on known facilities does not reach the target recall at a new facility. The Grad-CAM heatmaps of most correctly classified images lie on the features that the labels are based on, but for some clarifier classes they lie largely on the rim of the tank.

In the next chapter we will draw conclusions from these results, discuss the limitations of the project, and suggest future work.

Chapter 7

Summary and Conclusions

TODO: Write the summary and conclusions, and a section on future work. Ideas for future work:

* Implement the dashboard, where new images are classified, flagged components are relabelled by an operator, and the relabelled images are added to the training data.
* Use satellite data with more spectral bands (for example near-infrared) or a higher resolution, instead of RGB images from Google Earth.
* Review the aerobic zone labels in the same way as the clarifier labels, and have several operators label the same images to measure the agreement between them.

References

[1] TorchVision maintainers and contributors, "TorchVision: PyTorch's computer vision library," GitHub repository, 2016, version 0.28.0. https://github.com/pytorch/vision.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998, https://doi.org/10.1109/5.726791.

[3] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255, https://doi.org/10.1109/CVPR.2009.5206848.

[4] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015, https://doi.org/10.1007/s11263-015-0816-y.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, https://doi.org/10.1109/CVPR.2016.90.

[6] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2261–2269, https://doi.org/10.1109/CVPR.2017.243.

[7] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 6105–6114, https://proceedings.mlr.press/v97/tan19a.html.

[8] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11 966–11 976, https://doi.org/10.1109/CVPR52688.2022.01167.

[9] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141, https://doi.org/10.1109/CVPR.2018.00745.

[10] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv preprint arXiv:1606.08415, 2016, https://arxiv.org/abs/1606.08415.

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626, https://doi.org/10.1109/ICCV.2017.74.

[12] Department of Water and Sanitation, "Green Drop national report 2022," Department of Water and Sanitation, Pretoria, South Africa, Report, 2022, https://ws.dws.gov.za/IRIS/releases/GD22%20Report_NATIONAL_FINAL_16May22_MN%20web.pdf. Archived at https://web.archive.org/web/20220710190541/https://ws.dws.gov.za/IRIS/releases/GD22%20Report_NATIONAL_FINAL_16May22_MN%20web.pdf. Accessed 16 October 2026.

[13] J. C. Costa, D. P. Mesquita, A. L. Amaral, M. M. Alves, and E. C. Ferreira, "Quantitative image analysis for the characterization of microbial aggregates in biological wastewater treatment: a review," *Environmental Science and Pollution Research*, vol. 20, no. 9, pp. 5887–5912, 2013, https://doi.org/10.1007/s11356-013-1824-5.

[14] S. Borzooei, L. Scabini, G. Miranda, S. Daneshgar, L. Deblieck, O. Bruno, P. De Langhe, B. De Baets, I. Nopens, and E. Torfs, "Evaluation of activated sludge settling characteristics from microscopy images with deep convolutional neural networks and transfer learning," *Journal of Water Process Engineering*, vol. 64, p. 105692, 2024, https://doi.org/10.1016/j.jwpe.2024.105692.

[15] H. Satoh, Y. Kashimoto, N. Takahashi, and T. Tsujimura, "Deep learning-based morphology classification of activated sludge flocs in wastewater treatment plants," *Environmental Science: Water Research & Technology*, vol. 7, no. 2, pp. 298–305, 2021, https://doi.org/10.1039/D0EW00908C.

[16] J. Carballo Mato, S. González Vázquez, J. Fernández Águila, Á. Delgado Rodríguez, X. Lin, L. Garabato Gándara, J. Sobreira Seoane, and J. Silva Castro, "Foam segmentation in wastewater treatment plants," *Water*, vol. 16, no. 3, p. 390, 2024, https://doi.org/10.3390/w16030390.

[17] M. H. Gholizadeh, A. M. Melesse, and L. Reddi, "A comprehensive review on water quality parameters estimation using remote sensing techniques," *Sensors*, vol. 16, no. 8, p. 1298, 2016, https://doi.org/10.3390/s16081298.

[18] M. Lesiv, L. See, J. C. Laso Bayas, T. Sturn, D. Schepaschenko, M. Karner, I. Moorthy, I. McCallum, and S. Fritz, "Characterizing the spatial and temporal availability of very high resolution satellite imagery in Google Earth and Microsoft Bing Maps as a

source of reference data," *Land*, vol. 7, no. 4, p. 118, 2018, https://doi.org/10.3390/land7040118.

[19] Q. Hu, W. Wu, T. Xia, Q. Yu, P. Yang, Z. Li, and Q. Song, "Exploring the use of Google Earth imagery and object-based methods in land use/cover mapping," *Remote Sensing*, vol. 5, no. 11, pp. 6026–6042, 2013, https://doi.org/10.3390/rs5116026.

[20] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 145, pp. 197–209, 2018, https://doi.org/10.1016/j.isprsjprs.2018.01.004.

[21] H. Li, J. Zech, D. Hong, P. Ghamisi, M. Schultz, and A. Zipf, "Leveraging OpenStreetMap and multimodal remote sensing data with joint deep learning for wastewater treatment plants detection," *International Journal of Applied Earth Observation and Geoinformation*, vol. 110, p. 102804, 2022, https://doi.org/10.1016/j.jag.2022.102804.

[22] Y. Shuai, J. Xie, K. Lu, and Z. Chen, "Multi-attention network for sewage treatment plant detection," *Sustainability*, vol. 15, no. 7, p. 5880, 2023, https://doi.org/10.3390/su15075880.

[23] S. Randhawa, G. Randhawa, O. Sivak, J. Zech, M. Martin, A. Zipf, and Y. Li, "Multiscale multifeature vision learning for scalable and efficient wastewater treatment plant detection using hi-res satellite imagery and OSM," in *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Advances in Urban-AI (UrbanAI '23)*. New York, NY, USA: Association for Computing Machinery, 2023, pp. 10–21, https://doi.org/10.1145/3615900.3628772.

[24] C. Handan-Nader and D. E. Ho, "Deep learning to map concentrated animal feeding operations," *Nature Sustainability*, vol. 2, no. 4, pp. 298–306, 2019, https://doi.org/10.1038/s41893-019-0246-x.

[25] K. Nogueira, O. A. B. Penatti, and J. A. dos Santos, "Towards better exploiting convolutional neural networks for remote sensing scene classification," *Pattern Recognition*, vol. 61, pp. 539–556, 2017, https://doi.org/10.1016/j.patcog.2016.07.001.

[26] O. A. B. Penatti, K. Nogueira, and J. A. dos Santos, "Do deep features generalize from everyday objects to remote sensing and aerial scenes domains?" in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2015, pp. 44–51, https://doi.org/10.1109/CVPRW.2015.7301382.

[27] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann, and W. Brendel, "ImageNet-trained CNNs are biased towards texture; increasing shape bias improves

accuracy and robustness," in *International Conference on Learning Representations (ICLR)*, 2019, https://arxiv.org/abs/1811.12231.

[28] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018, https://doi.org/10.1016/j.neunet.2018.07.011.

[29] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2999–3007, https://doi.org/10.1109/ICCV.2017.324.

[30] T. Kattenborn, F. Schiefer, J. Frey, H. Feilhauer, M. D. Mahecha, and C. F. Dormann, "Spatially autocorrelated training and validation samples inflate performance assessment of convolutional neural networks," *ISPRS Open Journal of Photogrammetry and Remote Sensing*, vol. 5, p. 100018, 2022, https://doi.org/10.1016/j.ophoto.2022.100018.

[31] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, 2020, https://doi.org/10.1038/s42256-020-00257-z.

[32] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, "Unmasking Clever Hans predictors and assessing what machines really learn," *Nature Communications*, vol. 10, no. 1, p. 1096, 2019, https://doi.org/10.1038/s41467-019-08987-4.

[33] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015, https://doi.org/10.1371/journal.pone.0118432.

[34] A. Dutta and A. Zisserman, "The VIA annotation software for images, audio and video," in *Proceedings of the 27th ACM International Conference on Multimedia (MM '19)*. ACM, 2019, pp. 2276–2279, https://doi.org/10.1145/3343031.3350535.

[35] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826, https://doi.org/10.1109/CVPR.2016.308.

[36] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, https://doi.org/10.48550/arXiv.1912.01703.

[37] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015, https://doi.org/10.48550/arXiv.1412.6980.

[38] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631, https://doi.org/10.1145/3292500.3330701.

[39] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems*, vol. 24, 2011, pp. 2546–2554, https://proceedings.nips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf.

[40] J. Gildenblat and contributors, "PyTorch library for CAM methods," GitHub repository, 2021, version 1.5.7. https://github.com/jacobgil/pytorch-grad-cam.

Dataset Details

This appendix contains supporting material for Chapter 4: an example of the VIA2 annotation format, the leave-one-out split of the clarifiers, and the relabelling interface.

A.1 Annotation Format

An example entry of a VIA2 project file for an aerobic zone annotation is shown below. Each entry records the image filename, file size, and a list of regions, where each region specifies its shape, pixel coordinates, and class label.

```
"Atlantis 01_2019.jpg3469910": {
  "filename": "Atlantis 01_2019.jpg",
  "size": 3469910,
  "regions": [
    {
      "shape_attributes": {
        "name": "rect",
        "x": 2707, "y": 931,
        "width": 2839, "height": 1381
      },
      "region_attributes": { "aerobic zone": "Dysfunctional" }
    }
  ],
  "file_attributes": {}
}
```

A.2 Leave-one-out Split

A.3 Relabelling Interface

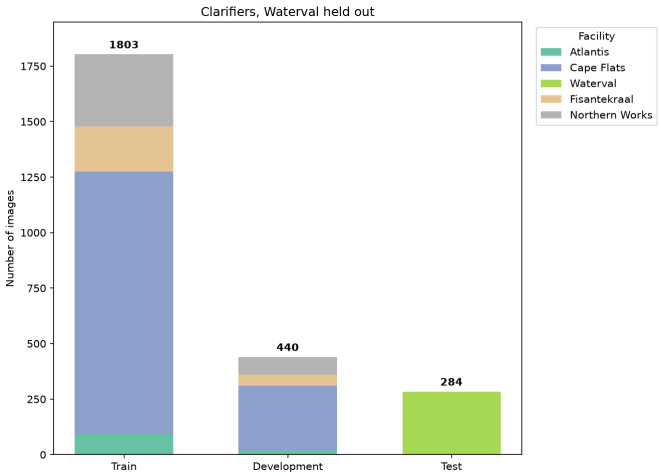


Figure A.1: Number of clarifier images from each facility in the training, development and test sets when Waterval is held out. The test set contains only Waterval, while the remaining four facilities are split into training and development sets.



Figure A.2: The relabelling interface, showing the original label and the per-class correction buttons.

Hyperparameter Search Trials

Section 5.5 describes the Optuna hyperparameter search used for the four pretrained models, which was run separately for the clarifiers and for the aerobic zones. This appendix lists every trial of these eight searches, so the results in that section can be checked in full without cluttering the main text with 160 rows of trial data. For each search, the winning trial (the one whose values were carried forward to training) is shaded.

B.1 Clarifiers

B.1.1 ResNet-18

Table B.1: All 20 clarifier Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	**LR**	β_1	**Step**	**Batch**	**Outcome**	**Dev. acc.**
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	49.64%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	84.58%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	56.39%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	82.89%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	72.53%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	83.86%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	85.78%
8	2	2048	9.08×10^{-6}	0.572	22	8	Pruned	–
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	82.89%
10	1	512	1.34×10^{-5}	0.797	11	32	Pruned	–
11	2	2048	3.77×10^{-4}	0.875	17	96	Completed	82.89%
12	1	512	7.37×10^{-3}	0.559	16	64	Pruned	–
13	2	2048	1.21×10^{-4}	0.622	23	96	Completed	80.00%
14	3	2048	5.82×10^{-3}	0.651	22	96	Pruned	–
15	1	512	4.65×10^{-4}	0.539	20	16	Completed	84.34%
16	2	2048	2.76×10^{-3}	0.774	14	96	Pruned	–
17	2	2048	2.49×10^{-3}	0.504	28	48	Pruned	–
18	1	2048	1.47×10^{-3}	0.977	20	32	Pruned	–
19	2	2048	9.46×10^{-3}	0.645	34	96	Pruned	–

B.1.2 EfficientNet-B0

Table B.2: All 20 clarifier Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	84.82%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	87.71%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	84.82%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	77.11%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	62.89%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	81.93%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Pruned	–
8	2	2048	9.08×10^{-6}	0.572	22	8	Pruned	–
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	84.34%
10	2	2048	6.20×10^{-3}	0.629	14	96	Pruned	–
11	1	512	1.39×10^{-3}	0.835	33	8	Pruned	–
12	0	2048	8.76×10^{-4}	0.671	30	96	Completed	88.43%
13	2	2048	4.06×10^{-4}	0.578	23	96	Completed	85.78%
14	0	2048	6.35×10^{-4}	0.708	31	96	Completed	87.71%
15	0	2048	1.69×10^{-3}	0.541	20	8	Pruned	–
16	0	2048	8.42×10^{-5}	0.794	17	24	Completed	86.99%
17	1	2048	1.69×10^{-3}	0.758	40	96	Pruned	–
18	0	2048	8.28×10^{-3}	0.735	23	128	Pruned	–
19	2	2048	9.51×10^{-3}	0.592	34	96	Pruned	–

B.1.3 DenseNet-121

Table B.3: All 20 clarifier Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	76.87%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	85.30%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	60.48%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	81.69%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	72.77%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	85.06%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	83.86%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	84.58%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	84.82%
10	2	2048	6.20×10^{-3}	0.629	14	96	Pruned	–
11	2	512	8.03×10^{-4}	0.562	14	48	Pruned	–
12	2	2048	5.50×10^{-4}	0.716	39	96	Completed	85.78%

continued on next page

Table B.3 (continued)

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
13	2	2048	4.06×10^{-4}	0.578	23	96	Completed	84.82%
14	2	2048	4.53×10^{-4}	0.746	33	96	Completed	83.13%
15	1	2048	2.30×10^{-5}	0.759	33	96	Pruned	–
16	2	2048	4.20×10^{-3}	0.564	33	64	Pruned	–
17	1	2048	1.69×10^{-3}	0.758	40	96	Pruned	–
18	2	2048	7.84×10^{-3}	0.774	22	128	Pruned	–
19	2	2048	1.38×10^{-4}	0.731	24	24	Completed	86.75%

B.1.4 ConvNeXt-Tiny

Table B.4: All 20 clarifier Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped before completion. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	42.89%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	43.86%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	43.86%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	86.51%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	75.90%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	86.75%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	88.67%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	88.43%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	86.51%
10	1	512	1.34×10^{-5}	0.797	11	32	Completed	86.02%
11	2	512	6.30×10^{-6}	0.541	28	48	Pruned	–
12	2	2048	5.26×10^{-6}	0.662	21	8	Completed	86.99%
13	2	512	2.85×10^{-4}	0.588	33	8	Pruned	–
14	2	2048	2.05×10^{-5}	0.672	5	8	Completed	87.95%
15	1	1024	9.93×10^{-5}	0.855	24	32	Completed	90.12%
16	1	2048	2.72×10^{-4}	0.831	25	32	Pruned	–
17	1	512	9.31×10^{-5}	0.927	20	32	Completed	89.40%
18	1	2048	1.16×10^{-3}	0.985	23	32	Pruned	–
19	1	1024	4.53×10^{-5}	0.822	22	32	Completed	88.92%

B.2 Aerobic Zones

B.2.1 ResNet-18

Table B.5: All 20 aerobic zone Optuna trials for ResNet-18, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	71.25%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	81.25%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	56.88%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	82.19%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	76.88%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	81.25%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	83.75%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	85.62%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	79.38%
10	2	2048	1.45×10^{-5}	0.571	30	24	Completed	84.69%
11	2	2048	1.31×10^{-6}	0.671	26	8	Pruned	–
12	2	2048	4.57×10^{-5}	0.713	37	24	Completed	85.94%
13	2	2048	1.90×10^{-5}	0.646	18	24	Completed	86.25%
14	2	2048	1.29×10^{-3}	0.591	36	24	Pruned	–
15	2	2048	1.82×10^{-4}	0.764	35	24	Completed	84.06%
16	0	2048	8.42×10^{-5}	0.794	17	24	Completed	86.56%
17	0	2048	2.84×10^{-4}	0.618	12	24	Completed	86.56%
18	0	2048	4.74×10^{-4}	0.643	8	24	Pruned	–
19	0	2048	3.94×10^{-5}	0.759	16	24	Completed	86.56%

B.2.2 EfficientNet-B0

Table B.6: All 20 aerobic zone Optuna trials for EfficientNet-B0, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	80.31%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	84.69%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	75.94%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	79.06%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	71.88%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	78.75%
6	2	512	3.06×10^{-3}	0.901	11	32	Completed	79.06%
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	82.19%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	79.38%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	80.94%
10	2	2048	6.20×10^{-3}	0.629	14	96	Completed	80.62%
11	2	2048	3.77×10^{-4}	0.875	17	96	Completed	82.50%
12	0	2048	3.46×10^{-4}	0.788	25	96	Completed	85.00%

continued on next page

Table B.6 (continued)

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
13	2	2048	4.06×10^{-4}	0.594	23	96	Completed	81.88%
14	0	2048	3.16×10^{-4}	0.805	28	96	Pruned	–
15	0	2048	4.95×10^{-4}	0.946	25	96	Completed	84.06%
16	2	2048	2.76×10^{-3}	0.774	14	96	Pruned	–
17	0	2048	3.14×10^{-3}	0.713	28	24	Completed	81.56%
18	2	2048	1.72×10^{-4}	0.750	22	16	Completed	88.12%
19	2	2048	5.15×10^{-5}	0.761	18	16	Completed	85.94%

B.2.3 DenseNet-121

Table B.7: All 20 aerobic zone Optuna trials for DenseNet-121, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	LR	β_1	Step	Batch	Outcome	Dev. acc.
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	75.94%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	78.44%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	48.75%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	81.88%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	76.56%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	83.44%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	83.12%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	85.31%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	82.19%
10	2	2048	1.45×10^{-5}	0.571	30	24	Completed	84.38%
11	2	2048	1.31×10^{-6}	0.671	26	8	Pruned	–
12	2	2048	4.57×10^{-5}	0.713	37	24	Completed	82.19%
13	2	2048	8.32×10^{-6}	0.565	20	24	Pruned	–
14	2	2048	3.42×10^{-4}	0.532	29	24	Completed	84.06%
15	2	1024	2.38×10^{-6}	0.540	32	48	Pruned	–
16	0	2048	2.36×10^{-5}	0.637	28	32	Completed	85.62%
17	0	2048	9.56×10^{-5}	0.543	21	32	Completed	84.69%
18	0	2048	1.39×10^{-4}	0.678	27	32	Completed	88.75%
19	0	2048	1.42×10^{-5}	0.645	25	32	Pruned	–

B.2.4 ConvNeXt-Tiny

Table B.8: All 20 aerobic zone Optuna trials for ConvNeXt-Tiny, in sampling order. *Pruned* trials were stopped early by the MedianPruner. The winning trial is shaded.

Trial	H	N	**LR**	β_1	**Step**	**Batch**	**Outcome**	**Dev. acc.**
0	1	512	2.92×10^{-3}	0.800	30	16	Completed	48.75%
1	2	2048	1.38×10^{-3}	0.600	23	96	Completed	48.75%
2	2	1024	4.34×10^{-3}	0.629	28	48	Completed	48.75%
3	1	2048	1.22×10^{-5}	0.914	17	64	Completed	82.81%
4	1	512	2.91×10^{-6}	0.931	27	96	Completed	75.94%
5	2	512	5.13×10^{-5}	0.513	8	48	Completed	87.19%
6	2	512	3.06×10^{-3}	0.901	11	32	Pruned	–
7	1	512	2.24×10^{-5}	0.971	16	32	Completed	85.00%
8	2	2048	9.08×10^{-6}	0.572	22	8	Completed	86.88%
9	3	512	2.31×10^{-4}	0.838	5	96	Completed	83.44%
10	3	512	1.06×10^{-5}	0.580	33	48	Pruned	–
11	2	512	4.04×10^{-5}	0.582	5	48	Pruned	–
12	2	2048	5.26×10^{-6}	0.588	21	8	Completed	85.31%
13	2	512	2.87×10^{-5}	0.727	9	48	Completed	85.62%
14	0	2048	1.14×10^{-6}	0.528	7	8	Pruned	–
15	2	2048	6.78×10^{-5}	0.512	27	8	Completed	91.56%
16	2	512	2.43×10^{-5}	0.700	29	48	Completed	89.69%
17	2	512	2.16×10^{-4}	0.546	32	48	Pruned	–
18	2	2048	1.42×10^{-4}	0.748	27	48	Completed	86.88%
19	2	512	1.06×10^{-5}	0.735	24	24	Pruned	–

Additional Results

This appendix contains the detailed results that are summarised in Chapter 6: the full multi-class results of every architecture, the per-facility results of the leave-one-facility-out experiment, the training curves and confusion matrices of the selected models, the augmentation experiment, and the binary results at the chosen thresholds. In every table, the best value in every column is shown in bold.

C.1 Experiment 1: Mixed-Facility Baseline

Table C.1: Clarifier test results of every architecture on the chronological split (Experiment 1). Macro-averaged scores give every class the same weight. The best value in every column is shown in bold.

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.776	0.929	0.734	0.733	0.732
EfficientNet-B0	0.780	0.927	0.679	0.715	0.691
DenseNet-121	0.785	0.926	**0.798**	**0.760**	**0.772**
ConvNeXt-Tiny	**0.838**	**0.949**	0.770	0.743	0.748
ResNet-18 (scratch)	0.713	0.904	0.650	0.704	0.667

Table C.1 shows that the macro F1 score of the pretrained clarifier models ranges from 0.691 (EfficientNet-B0) to 0.772 (DenseNet-121). ConvNeXt-Tiny has the highest accuracy (0.838) and mean AUC (0.949). EfficientNet-B0 has the third-highest accuracy but the lowest macro F1 score of the pretrained models, because of its low F1 score for the **Empty** class (Table C.2). ResNet-18 (scratch) has the lowest value in every column.

Table C.2: Per-class precision (P), recall (R) and F1 score of every clarifier architecture on the chronological split (Experiment 1).

	Dysfunctional			Empty			Functional			Scum		
Architecture	P	R	F1	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.738	0.792	0.764	0.606	0.588	0.597	0.829	0.726	0.774	0.763	0.827	0.794
EfficientNet-B0	0.766	0.750	0.758	0.315	0.500	0.386	0.812	**0.863**	0.837	0.823	0.746	0.782
DenseNet-121	0.750	0.844	0.794	**0.846**	**0.647**	**0.733**	0.837	0.688	0.756	0.760	**0.859**	0.806
ConvNeXt-Tiny	0.746	**0.885**	**0.810**	0.619	0.382	0.473	**0.851**	0.854	**0.852**	**0.865**	0.852	**0.858**
ResNet-18 (scratch)	**0.775**	0.823	0.798	0.352	0.559	0.432	0.687	0.829	0.751	0.786	0.607	0.685

Table C.2 shows that the **Empty** class has the lowest F1 score of every architecture, from 0.386 to 0.733. The F1 scores of the other three classes range from 0.756 to 0.858 for the pretrained models. The architectures differ most in the **Empty** precision, which ranges from 0.315 (EfficientNet-B0) to 0.846 (DenseNet-121).

Table C.3: Aerobic zone test results of every architecture on the chronological split (Experiment 1).

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.806	0.860	0.641	0.639	0.638
EfficientNet-B0	**0.857**	0.924	**0.748**	**0.733**	**0.740**
DenseNet-121	0.813	0.898	0.663	0.664	0.662
ConvNeXt-Tiny	0.850	**0.938**	0.724	0.724	0.724
ResNet-18 (scratch)	0.781	0.893	0.625	0.624	0.624

Table C.3 shows that EfficientNet-B0 and ConvNeXt-Tiny have the highest aerobic zone macro F1 scores (0.740 and 0.724), and the other two pretrained models have 0.638 and 0.662. The accuracy of every architecture is 0.12 to 0.17 higher than its macro F1 score, since the accuracy is dominated by the two large classes and the macro F1 score includes the **Suboptimal** class. ResNet-18 (scratch) has the lowest accuracy and macro F1 score.

Table C.4: Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture on the chronological split (Experiment 1).

	Dysfunctional			Functional			Suboptimal		
Architecture	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.817	0.932	0.871	0.911	0.818	0.862	0.196	0.167	0.180
EfficientNet-B0	**0.913**	0.884	**0.898**	0.875	**0.928**	0.901	**0.457**	**0.389**	**0.420**
DenseNet-121	0.854	**0.940**	0.895	0.903	0.811	0.854	0.232	0.241	0.236
ConvNeXt-Tiny	0.877	0.916	0.896	**0.925**	0.886	**0.905**	0.370	0.370	0.370
ResNet-18 (scratch)	0.852	0.811	0.831	0.849	0.875	0.862	0.172	0.185	0.179

Table C.4 shows that the **Dysfunctional** and **Functional** F1 scores range from 0.831 to 0.905, while the **Suboptimal** F1 score ranges from 0.179 to 0.420. The **Suboptimal** precision is at most 0.457, so for every architecture fewer than half of the aerobic zones classified as **Suboptimal** are **Suboptimal**.

C.2 Experiment 2: Leave-One-Facility-Out Evaluation

The training curves and confusion matrices are shown for the architecture with the highest mean macro F1 score for each component: ConvNeXt-Tiny for the clarifiers and DenseNet-121 for the aerobic zones.

Table C.5: Clarifier test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the five held-out facilities.

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.731 ± 0.089	0.891 ± 0.033	0.770 ± 0.101	0.648 ± 0.062	0.668 ± 0.068
EfficientNet-B0	0.805 ± 0.093	0.931 ± 0.034	0.806 ± 0.126	**0.749** ± 0.063	0.758 ± 0.098
DenseNet-121	0.770 ± 0.108	0.918 ± 0.025	0.767 ± 0.086	0.722 ± 0.073	0.724 ± 0.086
ConvNeXt-Tiny	**0.838** ± 0.063	**0.953** ± 0.014	**0.861** ± 0.079	0.748 ± 0.062	**0.775** ± 0.062
ResNet-18 (scratch)	0.735 ± 0.116	0.899 ± 0.049	0.720 ± 0.109	0.666 ± 0.085	0.662 ± 0.064

Table C.5 shows that ConvNeXt-Tiny has the highest mean accuracy, mean AUC, macro precision and macro F1 score, and the lowest standard deviation of the accuracy, mean AUC and macro F1 score across facilities. The standard deviation of the macro F1 score ranges from 0.062 (ConvNeXt-Tiny) to 0.098 (EfficientNet-B0). The macro precision is higher than the macro recall for every architecture.

Table C.6: Clarifier test accuracy and mean AUC of every architecture for each held-out facility.

Architecture	Atlantis	Cape Flats	Fisantekraal	Northern Works	Waterval	Mean
Accuracy						
ResNet-18	0.755	0.580	0.730	0.805	0.785	0.731
EfficientNet-B0	**0.873**	0.667	0.750	0.884	0.849	0.805
DenseNet-121	0.800	0.599	0.738	0.854	0.859	0.770
ConvNeXt-Tiny	0.827	**0.755**	**0.806**	**0.906**	**0.894**	**0.838**
ResNet-18 (scratch)	0.800	0.537	0.726	0.825	0.785	0.735
Mean AUC						
ResNet-18	0.935	0.852	0.910	0.873	0.884	0.891
EfficientNet-B0	0.965	0.882	0.915	**0.958**	0.937	0.931
DenseNet-121	0.941	0.876	0.926	0.926	0.920	0.918
ConvNeXt-Tiny	**0.966**	**0.938**	**0.940**	0.953	**0.967**	**0.953**
ResNet-18 (scratch)	0.927	0.814	0.908	0.912	0.935	0.899

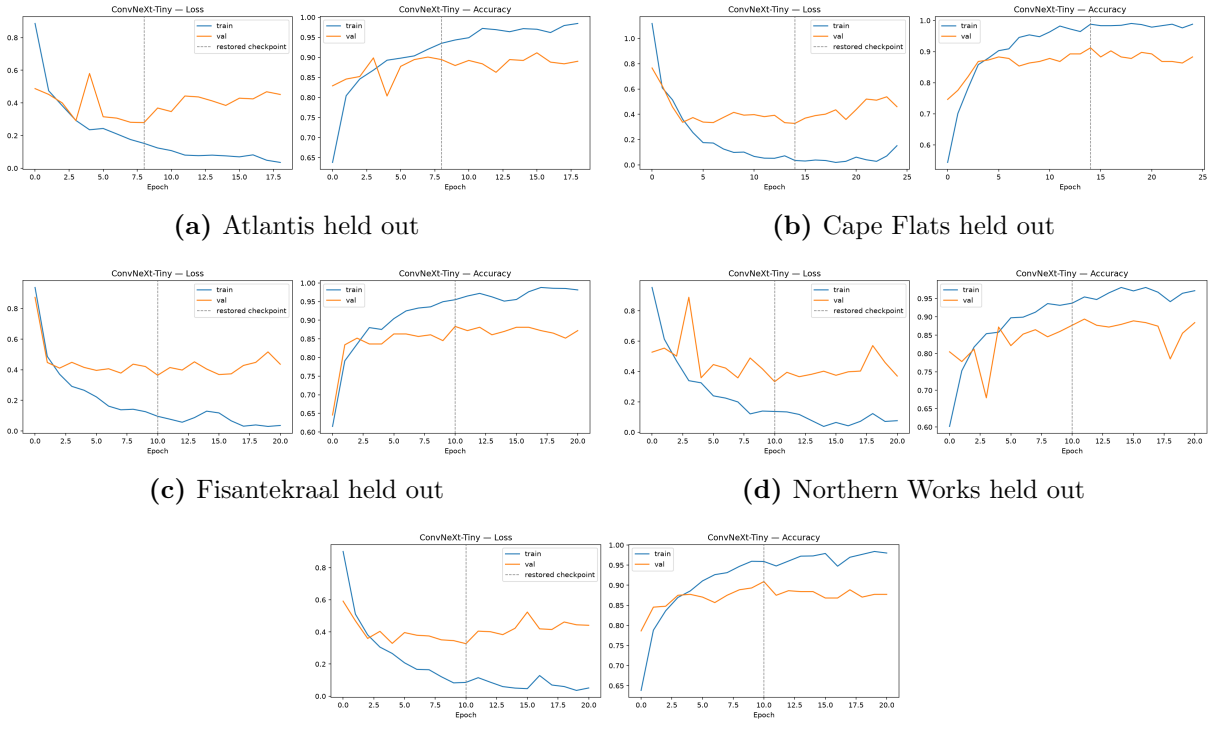
Table C.6 shows that Cape Flats has the lowest accuracy and the lowest mean AUC of every architecture. The accuracy at Cape Flats ranges from 0.537 to 0.755, against 0.726

to 0.906 at the other facilities. ConvNeXt-Tiny has the highest accuracy at four of the five facilities.

Table C.7: Per-class precision (P), recall (R) and F1 score of every clarifier architecture in Experiment 2, averaged over the held-out facilities at which the class occurs.

	Dysfunctional			**Empty**			**Functional**			**Scum**		
Architecture	P	R	F1	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.743	0.623	0.675	0.821	0.461	0.553	0.722	0.695	0.667	0.785	0.756	0.745
EfficientNet-B0	0.772	0.661	0.710	0.729	**0.662**	**0.649**	**0.820**	0.818	0.806	0.848	0.802	0.809
DenseNet-121	0.786	0.728	0.754	0.634	0.560	0.565	0.780	0.777	0.749	0.823	0.763	0.773
ConvNeXt-Tiny	**0.871**	0.710	**0.776**	**0.868**	0.505	0.598	0.819	**0.879**	**0.836**	**0.872**	**0.833**	**0.843**
ResNet-18 (scratch)	0.703	**0.772**	0.729	0.615	0.361	0.423	0.726	0.751	0.709	0.787	0.717	0.735

Table C.7 shows that the **Empty** class has the lowest recall for four of the five architectures, from 0.361 to 0.560. For EfficientNet-B0 it is 0.662, against a **Dysfunctional** recall of 0.661. The **Empty** precision is higher than the **Empty** recall for every architecture, so the main error is that **Empty** clarifiers are missed, not that other clarifiers are classified as **Empty**.



(a) Atlantis held out

(b) Cape Flats held out

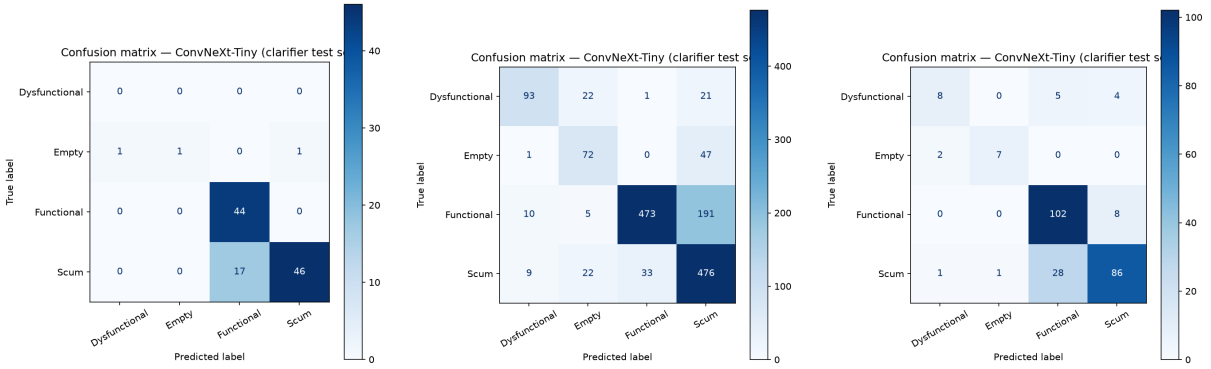
(c) Fisantekraal held out

(d) Northern Works held out

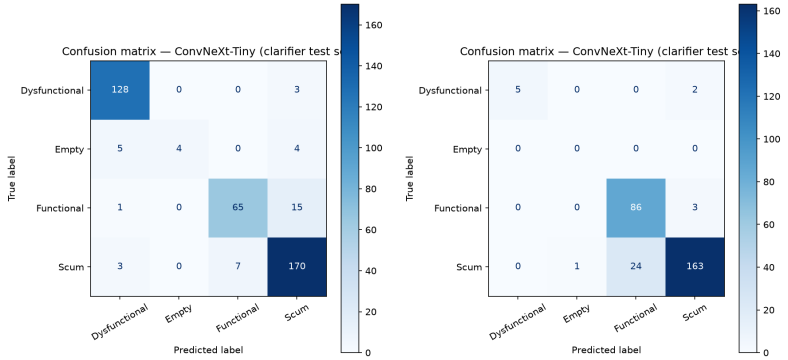
(e) Waterval held out

Figure C.1: Training and development loss and accuracy of the ConvNeXt-Tiny clarifier model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint.

Figure C.1 shows that the development loss of every fold stops decreasing after 9 to 15 epochs, and training stops 10 epochs later, well before the limit of 100 epochs. After the restored checkpoint, the training loss continues to decrease while the development loss stays level or increases. At the checkpoint, the development accuracy is about 0.88 to 0.91. The development set is drawn from the training facilities, and the test accuracy at the held-out facility (0.755 to 0.906, Table C.6) is lower for four of the five folds.



(a) Atlantis **(b)** Cape Flats **(c)** Fisantekraal



(d) Northern Works **(e)** Waterval

Figure C.2: Confusion matrices of the ConvNeXt-Tiny clarifier model on each held-out facility.

Figure C.2 shows that the most common mistake at every facility is a confusion between **Scum** and **Functional**. At Atlantis, Fisantekraal and Waterval, 17 of 63, 28 of 116 and 24 of 188 **Scum** clarifiers are classified as **Functional**. At Cape Flats and Northern Works the direction is reversed: 191 of 679 and 15 of 81 **Functional** clarifiers are classified as **Scum**. At Cape Flats, 47 of the 120 **Empty** clarifiers are also classified as **Scum**.

Table C.8 shows that DenseNet-121 has the highest mean accuracy, mean AUC, macro recall and macro F1 score, and ConvNeXt-Tiny the highest macro precision. The standard deviation of the macro F1 score ranges from 0.033 (DenseNet-121) to 0.194 (ConvNeXt-Tiny), and that of the accuracy from 0.052 (ResNet-18) to 0.255 (EfficientNet-B0). The accuracy of ResNet-18 (scratch) is 0.389, which is 0.140 below the lowest accuracy of a pretrained model.

Table C.8: Aerobic zone test results of every architecture in the leave-one-facility-out experiment (Experiment 2), given as the mean ± standard deviation across the three held-out facilities.

Architecture	Accuracy	Mean AUC	Macro precision	Macro recall	Macro F1
ResNet-18	0.598 ± 0.052	0.694 ± 0.057	0.438 ± 0.070	0.517 ± 0.029	0.435 ± 0.081
EfficientNet-B0	0.529 ± 0.255	0.705 ± 0.054	0.487 ± 0.051	0.484 ± 0.040	0.413 ± 0.130
DenseNet-121	**0.628** ± 0.096	**0.763** ± 0.025	0.498 ± 0.025	**0.541** ± 0.028	**0.495** ± 0.033
ConvNeXt-Tiny	0.540 ± 0.219	0.758 ± 0.088	**0.514** ± 0.114	0.537 ± 0.116	0.451 ± 0.194
ResNet-18 (scratch)	0.389 ± 0.155	0.624 ± 0.023	0.441 ± 0.029	0.399 ± 0.028	0.331 ± 0.101

Table C.9: Aerobic zone test accuracy and mean AUC of every architecture for each held-out facility.

Architecture	Atlantis	Cape Flats	Waterval	Mean
Accuracy				
ResNet-18	**0.619**	0.538	0.635	0.598
EfficientNet-B0	0.611	0.243	0.734	0.529
DenseNet-121	0.575	**0.570**	**0.740**	**0.628**
ConvNeXt-Tiny	0.593	0.300	0.728	0.540
ResNet-18 (scratch)	0.469	0.210	0.487	0.389
Mean AUC				
ResNet-18	0.697	0.636	0.750	0.694
EfficientNet-B0	0.715	0.647	0.753	0.705
DenseNet-121	0.736	**0.770**	0.783	**0.763**
ConvNeXt-Tiny	**0.820**	0.656	**0.796**	0.758
ResNet-18 (scratch)	0.645	0.628	0.600	0.624

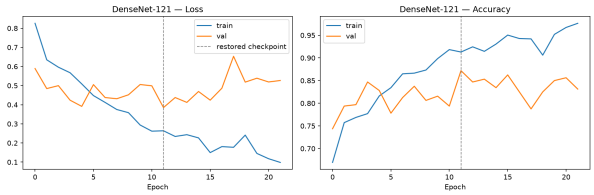
Table C.9 shows that Cape Flats has the lowest accuracy of every architecture, from 0.210 to 0.570. Most aerobic zones at Cape Flats are **Dysfunctional** (562 of 766, Figure C.4), so the accuracy there depends mainly on the **Dysfunctional** recall. EfficientNet-B0 and ConvNeXt-Tiny, the pretrained models with the lowest accuracy at Cape Flats (0.243 and 0.300), also have the largest standard deviation of the accuracy in Table C.8. The mean AUC does not follow the same pattern: for DenseNet-121, the AUC at Cape Flats is higher than at Atlantis.

Table C.10 shows that the **Suboptimal** recall ranges from 0.072 to 0.294 and the **Suboptimal** F1 score from 0.064 to 0.203. The **Dysfunctional** recall ranges from 0.244 to 0.726, against 0.811 to 0.940 in Experiment 1 (Table C.4).

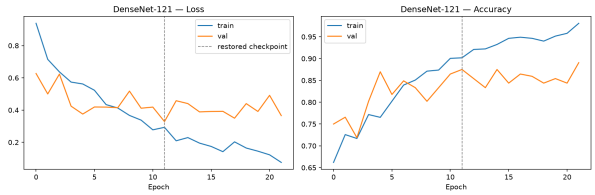
Figure C.3 shows that the development loss of every fold stops decreasing after 8 to 12 epochs, and training stops 10 epochs later. The development loss varies more from epoch to epoch than for the clarifiers. At the checkpoint, the development accuracy is about 0.79 to 0.87, while the test accuracy at the held-out facility is 0.570 to 0.740 (Table C.9). The development accuracy therefore overestimates the accuracy at a new facility.

Table C.10: Per-class precision (P), recall (R) and F1 score of every aerobic zone architecture in Experiment 2, averaged over the three held-out facilities.

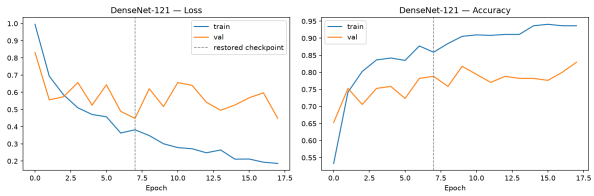
	Dysfunctional			Functional			Suboptimal		
Architecture	P	R	F1	P	R	F1	P	R	F1
ResNet-18	0.609	**0.726**	0.616	0.647	0.752	0.625	0.057	0.072	0.064
EfficientNet-B0	0.703	0.458	0.451	0.628	**0.796**	**0.669**	0.130	0.198	0.118
DenseNet-121	0.696	0.712	**0.680**	**0.675**	0.728	0.664	0.124	0.184	0.141
ConvNeXt-Tiny	**0.712**	0.593	0.550	0.639	0.793	0.599	**0.192**	0.224	**0.203**
ResNet-18 (scratch)	0.637	0.244	0.280	0.558	0.659	0.537	0.128	**0.294**	0.177



(a) Atlantis held out

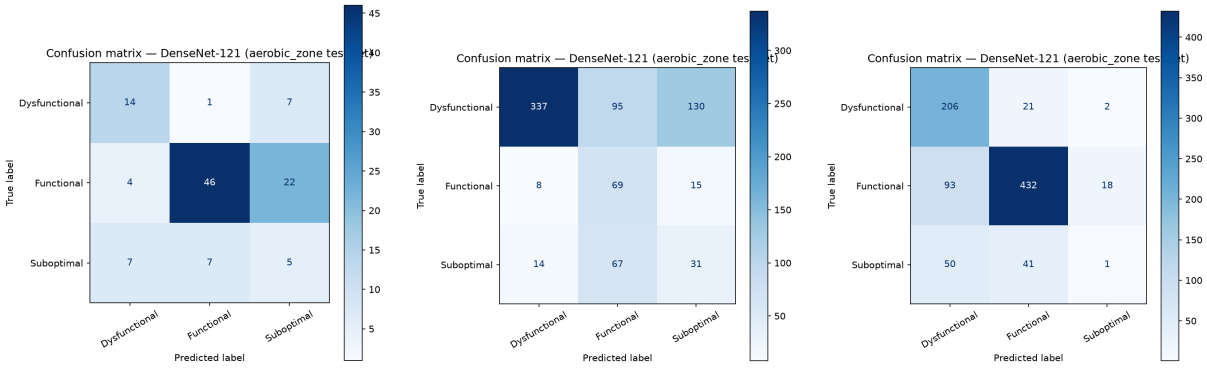


(b) Cape Flats held out



(c) Waterval held out

Figure C.3: Training and development loss and accuracy of the DenseNet-121 aerobic zone model for every leave-one-facility-out fold. The dashed line marks the epoch of the restored checkpoint.



(a) Atlantis **(b)** Cape Flats **(c)** Waterval

Figure C.4: Confusion matrices of the DenseNet-121 aerobic zone model on each held-out facility.

Figure C.4 shows that the most common mistake differs between facilities. At Atlantis, 22 of the 72 **Functional** zones are classified as **Suboptimal**. At Cape Flats, 130 and 95 of the 562 **Dysfunctional** zones are classified as **Suboptimal** and **Functional**, and 67 of the 112 **Suboptimal** zones as **Functional**. At Waterval, 93 of the 543 **Functional** zones are classified as **Dysfunctional**, and only 1 of the 92 **Suboptimal** zones is classified correctly.

C.3 Effect of Augmentation

Table C.11 compares the chronological-split results of every architecture trained with and without the augmentations described in Section 5.3.

Table C.11: Test results on the chronological split with (Aug.) and without (None) augmentation during training. Δ is the difference in percentage points, where a positive value means that augmentation helped.

	Accuracy			**Macro F1**			**Mean AUC**	
Architecture	Aug.	None	Δ	Aug.	None	Δ	Aug.	None
Clarifiers								
ResNet-18	0.776	0.745	+3.1	0.732	0.684	+4.8	0.929	0.913
EfficientNet-B0	0.780	0.789	-0.9	0.691	0.721	-3.0	0.927	0.921
DenseNet-121	0.785	0.777	+0.8	0.772	0.703	+6.9	0.926	0.919
ConvNeXt-Tiny	0.838	0.798	+4.0	0.748	0.736	+1.2	0.949	0.950
ResNet-18 (scratch)	0.713	0.655	+5.8	0.667	0.605	+6.2	0.904	0.857
Aerobic zones								
ResNet-18	0.806	0.810	-0.4	0.638	0.599	+3.9	0.860	0.885
EfficientNet-B0	0.857	0.808	+4.9	0.740	0.641	+9.9	0.924	0.849
DenseNet-121	0.813	0.852	-3.9	0.662	0.693	-3.1	0.898	0.925
ConvNeXt-Tiny	0.850	0.843	+0.7	0.724	0.744	-2.0	0.938	0.940
ResNet-18 (scratch)	0.781	0.776	+0.5	0.624	0.584	+4.0	0.893	0.811

Table C.11 shows that augmentation increased the clarifier accuracy and macro F1 score of four of the five models, all except EfficientNet-B0. For the aerobic zones, it increased the macro F1 score of three models, but decreased the accuracy and macro F1 score of DenseNet-121 by 3.9 and 3.1 percentage points. The largest increases in macro F1 score are 9.9 percentage points for the EfficientNet-B0 aerobic zones and 6.9 for the DenseNet-121 clarifiers.

C.4 Binary (OK / Not-OK) Results

Table C.12 gives the threshold chosen on the development set for every architecture, with the test recall of the Not-OK class at that threshold. Tables C.13 and C.14 give the recall and specificity for every held-out facility at the threshold of that fold.

Table C.12: Threshold t on the Not-OK score chosen for a 95% recall on the development set, with the test recall of the Not-OK class at that threshold. For the leave-one-facility-out split, every fold has its own threshold, and the threshold and recall are averaged across held-out facilities. The specificity and precision at these thresholds are given in Table 6.3 (Section 6.3).

	Chronological		**Leave-one-facility-out**	
Architecture	t	Recall	t	Recall
Clarifiers				
ResNet-18	0.244	0.974	0.280	0.867
EfficientNet-B0	0.194	0.961	0.328	0.902
DenseNet-121	0.309	0.959	0.309	0.897
ConvNeXt-Tiny	0.126	0.968	0.353	0.886
ResNet-18 (scratch)	0.140	0.964	0.238	0.916
Aerobic zones				
ResNet-18	0.541	0.931	0.312	0.846
EfficientNet-B0	0.257	0.941	0.400	0.740
DenseNet-121	0.585	0.924	0.387	0.864
ConvNeXt-Tiny	0.313	0.950	0.552	0.692
ResNet-18 (scratch)	0.455	0.881	0.475	0.719

Table C.12 shows that the chosen threshold differs between architectures, from 0.126 to 0.585 on the chronological split and from 0.238 to 0.552 as a mean across folds in the leave-one-facility-out experiment. The thresholds of the individual folds range from 0.114 to 0.591 for the clarifiers and from 0.253 to 0.834 for the aerobic zones.

Table C.13: Recall and specificity of the Not-OK class for each held-out facility at the clarifier threshold chosen on the development set of that fold, with the lowest recall and the standard deviation of the recall across facilities.

Architecture	**Atlantis**	**Cape Flats**	**Fisante-kraal**	**Northern Works**	**Waterval**	**Min.**	**Std.**
Recall							
ResNet-18	0.667	0.987	0.732	0.988	0.959	0.667	0.155
EfficientNet-B0	0.818	0.969	0.768	0.981	0.974	0.768	0.101
DenseNet-121	0.788	0.990	0.803	0.963	0.944	0.788	0.095
ConvNeXt-Tiny	0.758	0.956	0.810	0.985	0.923	0.758	0.098
ResNet-18 (scratch)	0.848	0.962	0.831	1.000	0.938	0.831	0.073
Specificity							
ResNet-18	0.955	0.187	0.818	0.247	0.494		
EfficientNet-B0	0.977	0.297	0.891	0.679	0.663		
DenseNet-121	0.977	0.236	0.845	0.741	0.753		
ConvNeXt-Tiny	1.000	0.704	0.882	0.790	0.854		
ResNet-18 (scratch)	0.909	0.174	0.700	0.049	0.708		

Table C.13 shows that the lowest clarifier recall of every architecture occurs at Atlantis or Fisantekraal, and ranges from 0.667 to 0.831. At these two facilities the specificity is 0.700 to 1.000, so the threshold flags too few components there. At Cape Flats and Northern Works the recall is at least 0.956, and at Cape Flats the specificity is at most 0.297 for every architecture except ConvNeXt-Tiny (0.704), so most OK clarifiers there are flagged.

Table C.14: Recall and specificity of the Not-OK class for each held-out facility at the aerobic zone threshold chosen on the development set of that fold, with the lowest recall and the standard deviation of the recall across facilities.

Architecture	Atlantis	Cape Flats	Waterval	Min.	Std.
Recall					
ResNet-18	0.976	0.760	0.804	0.760	0.114
EfficientNet-B0	0.902	0.795	0.523	0.523	0.195
DenseNet-121	0.878	0.935	0.779	0.779	0.079
ConvNeXt-Tiny	0.951	0.447	0.679	0.447	0.253
ResNet-18 (scratch)	0.829	0.782	0.545	0.545	0.152
Specificity					
ResNet-18	0.458	0.663	0.599		
EfficientNet-B0	0.500	0.685	0.917		
DenseNet-121	0.444	0.554	0.808		
ConvNeXt-Tiny	0.667	0.978	0.849		
ResNet-18 (scratch)	0.472	0.685	0.516		

Table C.14 shows that the lowest aerobic zone recall ranges from 0.447 to 0.779, and occurs at Waterval for three of the five architectures. The target of 95% is reached in only 2 of the 15 combinations of architecture and facility. DenseNet-121 has the smallest standard deviation of the recall (0.079, against 0.114 to 0.253) and the highest lowest recall. ConvNeXt-Tiny, which has the highest binary AUC in Experiment 2 (Table 6.3 in Section 6.3), has a recall of 0.447 at Cape Flats.